Prediction-Based Dual-Stage Distributed Task Allocation for Maximizing Assignments in Search and Rescue Scenarios

Yulin Li, Chunyan Zhang, *Member, IEEE*, Qiuyang Fang, Xudong Zhao, *Senior Member, IEEE*, and Jianlei Zhang, *Member, IEEE*

***Abstract*—Multi-robot systems provide an effective solution for executing complex tasks in dynamic environments; however, achieving efficient task allocation–optimizing the division of labor among robots for satisfactory performance–poses a significant challenge. This study introduces a prediction-based dual-stage distributed task allocation (PDTA) method for distributed multi-robot systems, with a particular focus on search and rescue (SAR) scenarios. In the first stage, the method utilizes a performance impact algorithm framework to generate an initial solution. It incorporates a task bid prediction mechanism during the task removal process to reduce invalid bids and expedite the convergence rate of the algorithm. The cost function also integrates task deadlines, enabling robots to allocate time more effectively and complete additional tasks within fuel constraints. In the second stage, a local search mechanism refines the initial solution to improve the quality of task allocation and the number of completed tasks. Monte Carlo simulations in SAR scenarios, considering task deadlines and fuel limitations, validate the effectiveness of this approach. The results demonstrate significant advantages over existing distributed task allocation methods, particularly in iteration efficiency and the number of rescued survivors.**

***Index Terms*—Task allocation, distributed multi-robot systems, search and rescue scenarios.**

I. Introduction

With the rapid advancements in robotics technology, mobile robots have significantly improved in functional capabilities and operational stability, and multi-robot systems consisting of multiple homogeneous or heterogeneous robots, have emerged as a viable solution for task execution in intricate and volatile environments [1], [2], [3]. By facilitating communication and collaboration among robots, multi-robot systems improve task execution efficiency compared to single-robot systems, thus garnering significant scholarly interest [4], [5], [6], [7], [8]. A pivotal issue in deploying these collaborative systems is multi-robot task allocation (MRTA), which involves strategically assigning tasks to agents to optimize performance [9], [10], [11], [12]. A wide range of effective methods have been proposed for the MRTA problem [13], including market-based approaches [14], optimization-based formulations [15], and bio-inspired [16] or learning-based strategies [17], [18]. These task allocation methods have been successfully applied in various real-world domains, such as regional exploration [19], environmental monitoring [20], topographic mapping, and logistics [21].

Unlike coordination in other robotic team applications, SAR missions face distinct challenges stemming from the dynamic, time-sensitive, and inherently unpredictable nature of disaster environments, which demand efficient resource allocation [22], [23], [24]. For instance, deploying a heterogeneous swarm of drones in emergency rescue operations must contend with constraints such as limited battery capacity and payload capacity [25], [26]. Furthermore, the survivability of victims deteriorates over time, requiring the strictness of the deadlines to deliver critical supplies [27], [28]. Consequently, task allocation must consider the balance between priority setting, temporal constraints, and energy limitations to ensure mission success [29], [30], [31]. The objectives in such scenarios are to maximize the number of successful rescues while minimizing survivors' waiting time. Although existing literature introduces methods such as robot coalition formation [32], [33], [34], [35], which rely on relatively stable communication and sufficient agent availability, these methods are no longer applicable in real-world sudden disasters, where robotic resources are severely limited and communication is unreliable. In time-critical scenarios, the robot team may be unable to complete all tasks, necessitating efficient resource allocation to prioritize critical ones [36], [37].

This study addresses the challenge of optimizing task allocation in SAR scenarios under constrained resources, where each robot can be assigned multiple tasks, each of which must be performed by a type-compatible robot and completed within its deadline. Following the established classification framework that distinguishes problems through robotic capabilities, task requirements, and temporal relevance [38], [39], we formally characterize this scenario as an MT-SR-IA problem. ("MT"

Received 22 August 2025; revised 16 October 2025; accepted 16 October 2025. Date of publication 23 October 2025; date of current version 11 December 2025. This work was supported by the National Natural Science Foundation of China under Grant 62473211 and Grant 62573239. Recommended for acceptance by Dr. Dusit Niyato. *(Corresponding author: Jianlei Zhang.)*

Yulin Li, Chunyan Zhang, Qiuyang Fang, and Jianlei Zhang are with the College of Artificial Intelligence, Nankai University, Tianjin 300381, China (e-mail: yulinlifight@mail.nankai.edu.cn; zhcy@nankai.edu.cn; qiuyangfang@mail.nankai.edu.cn; jianleizhang@nankai.edu.cn).

Xudong Zhao is with the Key Laboratory of Intelligent Control and Optimization for Industrial Equipment, Ministry of Education, Dalian University of Technology, Dalian 116024, China (e-mail: xdzhaohit@gmail.com).

Digital Object Identifier 10.1109/TNSE.2025.3624250

2327-4697 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

TABLE I
COMPARISON OF OUR WORK WITH EXISTING LITERATURE

Works	Adaptability	Convergence Speed	Solution Quality	Main Objective
CBBA[77]	High	Fast	Low	Maximizing total value
PI[78]	Medium	Medium	Medium	Minimizing total arrived time
PIsoftmax[79]	High	Slow	High	Maximizing task completion
PImaxAss[80]	Low	Slow	High	Maximizing task completion
PIDopt[81]	Medium	Slow	High	Maximizing task completion
MTDS[82]	Medium	Medium	Medium	Maximizing total value
PDTA	High	Fast	High	Maximizing task completion

indicates that robots can be assigned multiple tasks to be executed in sequence, "SR" means that each task requires exactly one robot to accomplish it, and "IA" denotes instantaneous assignment.) This scenario presents a vast solution space with significant temporal dependencies between tasks, posing challenges in achieving effective task allocation, and thus motivating our research. Based on these observations, this paper introduces a prediction-based dual-stage distributed task allocation method designed to develop an effective task allocation decision framework tailored for SAR environments. The proposed algorithm is particularly suited to SAR scenarios, but it can also be extended to other situations with similar conditions and constraints. Main contributions are as follows:

1) Introduction of a Task Prediction Mechanism: Unlike existing approaches, this study introduces a task prediction mechanism during the task removal phase, aimed at forecasting competitors' bids. This procedure significantly mitigates the effects of invalid bids on the allocation process, thereby reducing the number of iterations required for algorithmic convergence. Moreover, it enhances the algorithm's robustness, especially in low-bandwidth communication environments.
2) Design of a Deadline-Aware Cost Function: A deadline-aware cost function is developed to optimize task assignments by effectively incorporating task deadlines. By prioritizing tasks with earlier deadlines, this function avoids the premature insertion of tasks with later deadlines that might hinder the assignment of other critical tasks. This strategic prioritization maximizes the number of tasks assigned, optimizing resource usage and enhancing overall system efficacy.
3) Incorporation of a Local Search Mechanism: To further enhance performance, a local search mechanism is integrated into the framework, aiming to expand the solution space and avoid overlooking critical rescue tasks. Simulation results indicate that this approach consistently outperforms several baseline algorithms in SAR scenarios, demonstrating effectiveness in task allocation optimization.

The structure of the paper is organized as follows: Section II reviews the related works and presents a comparison with existing literature in Table I. Section III describes the SAR task allocation problem and presents the corresponding optimization model. Section IV introduces the proposed distributed task allocation algorithm. Section V discusses the simulation results. Finally, Section VI concludes the paper.

II. RELATED WORK

A. Task Allocation in Search and Rescue Scenarios

In post-disaster SAR scenarios, robot teams are required to collaboratively perform diverse tasks, such as restoring communication networks [40], [41], delivering materials [42], [43], [44], and conducting environmental monitoring [45], [46]. This environment demands that multi-robot systems achieve collaboration and decision-making under incomplete information and constrained resources, which has motivated numerous studies on effective task allocation mechanisms in this scenario [47], [48], [49].

When deploying heterogeneous robotic systems in real environments, each robot is typically equipped with diverse sensing, communication, and computing resources, enabling robot teams to exploit resource complementarity and improve task completion rates [35], [50], [51]. Leveraging this complementarity, the resource—task allocation problem is modeled as an overlapping coalition game, and the preference–gravity-guided tabu search algorithm has been proposed to accelerate coalition formation of robot teams in post-disaster scenarios [52]. In addition, some studies have investigated post-disaster network resource allocation [53]. For example, [54] investigated task allocation in uncrewed aerial vehicles (UAVs)–assisted network reconstruction, in which UAVs acted as relay stations to restore disrupted communication links. Similarly, [55] examined task allocation in UAV-UGV cooperative networks, with multiple unmanned ground vehicles (UGVs) collecting disaster information and UAVs assisting in relaying data, enhancing communication robustness of emergency communication networks through air-ground collaboration.

On the other hand, robot teams are required to operate under incomplete task information, making the elimination of uncertainty's impact an important research aspect [56], [57], [58]. To handle uncertainty in task type, self-learning-based coalition formation methods have been proposed, enabling robots to iteratively update their strategies and progressively infer task types. The main idea is to use Bayesian belief fusion so that robots can refine their understanding of tasks through repeated observations, thereby confirming task type [59]. In response to uncertain task requirements and agent capabilities, risk-adaptive task allocation strategies have been developed by incorporating conditional value at risk into the cost function to maximize mission success rate [30]. In addition, during disaster relief operations, robot failures may occur due to complex terrain or

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

communication loss. To ensure the robustness of system, [60] proposed fault-tolerant decentralized cluster coalition formation frameworks, enabling effective cooperation even under individual robot failures. Although extensive research has been conducted on various aspects of post-disaster task allocation, task allocation strategies that simultaneously address complex constraints such as deadlines, resource limitations, and task-type requirements remain insufficient.

B. Centralized Versus Distributed Frameworks

System frameworks for multi-robot systems are generally categorized into two types based on the network communication structure: centralized and distributed. In centralized systems, a central node consolidates information from the entire network to optimize the global objective using comprehensive data. The task allocation problem in SAR missions is akin to the robot routing problem with time windows (VRPTW) and is classified as NP-hard [61], which motivates researchers to develop optimization-based approaches to address this issue. Several studies have addressed static task allocation problems by employing a central coordinator to derive suboptimal solutions using heuristic algorithms [62], [63], [64], [65] or mixed-integer programming methods [66], [67]. Although centralized systems can achieve optimal solutions, their computational time grows exponentially with increasing problem complexity, imposing substantial computational demands on the central node. Moreover, centralized systems are vulnerable to single points of failure: if the central node fails, the entire system's functionality is compromised. In contrast, distributed structure operates without a central agent, with all robots participating in task allocation and computation. Each robot communicates only with neighbor robots, avoiding reliance on a central control system and minimizing communication bottlenecks and delays. Additionally, distributed systems ensure task execution independence, allowing the system to continue functioning even if some robots fail, thereby significantly enhancing robustness, particularly in complex and hazardous environments. As a result, for practical applications—especially those involving large-scale task allocation or requiring high reliability—distributed task allocation methods are preferred [68], [69], [70], [71], [72].

C. Auction-Based Task Allocation Algorithm

In distributed task allocation frameworks, the auction algorithm is a prevalent approach that aligns with market trading principles [14]. In this algorithm, each agent submits bids for tasks, with the auctioneer collecting these bids from all agents and assigning the task to the agent with the lowest bid [73]. This method decomposes computation across individual robots, leveraging local perception data to achieve decentralized computing and execution. As a result, it reduces reliance on a centralized node, thereby enhancing the robustness and computational efficiency of the multi-robot system. Building on previous work, researchers have eliminated the need for a central auctioneer by implementing a message-passing mechanism, thus enabling fully distributed task allocation [74]. Furthermore, in addressing complex multi-task allocation problems, a combinatorial auction method has been developed [75]. In this approach, robots submit bids on bundles of tasks rather than individual tasks [76], allowing for more efficient management of task interdependencies and yielding a more optimal overall task allocation. However, a notable challenge with this method is that the number of possible task bundles increases exponentially with the number of tasks, leading to significantly heightened computational complexity. To effectively address the distributed multi-robot task allocation problem, researchers introduced the consensus-based bundle algorithm (CBBA) [77]. This algorithm constructs and continuously updates a single bundle during the auction procedure, reaching consensus on the winning bid through a conflict resolution phase, ultimately producing a conflict-free solution. Various CBBA algorithm variants have been designed for different application scenarios.

D. Performance Impact Algorithm

The performance impact (PI) algorithm was presented particularly to address task allocation in SAR contexts [78]. Unlike the CBBA algorithm, which focuses on minimizing local costs calculated independently by each robot, the PI algorithm introduces the concept of significance, a metric intended to optimize the overall objective function—namely, the total arrival time of all robots—more directly. Recently, the PIsoftmax algorithm has integrated softmax and ε-greedy strategies into the PI framework. This integration expands the solution search space, mitigates the PI algorithm's tendency to converge to local optima, and extends its applicability to dynamic task allocation scenarios [79], [83]. Simulation results show that the PIsoftmax algorithm can reduce the average task execution time by up to 9% compared to the baseline PI algorithm [80]. Whereas the aforementioned algorithms primarily aim to minimize total arrival time, post-disaster scenarios prioritize the successful rescue of survivors as the main objective. Consequently, the PImaxAss algorithm extends the initial solution generated by the PI algorithm, reallocating tasks within the existing scheduling space. This allows robots to dynamically adjust task allocation to increase the number of successful tasks [80]. Recently, the PIDopt algorithm, an extension of the PImaxAss approach, further enhances task allocation by introducing a search mechanism to optimize assignments. This significantly improves task completion rates and allocation efficiency [81]. Meanwhile, the MTDS algorithm with task removal reasoning and a deadlock avoidance mechanism actively resolves conflicts, accelerates convergence, and prevents the system from falling into infinite loops of repeated priority swapping [82]. However, due to the necessity of rescheduling task lists, these methods frequently require more time and iterations to reach convergence, or they fail to maximize the number of assigned tasks, potentially constraining its performance in SAR task allocation especially in dynamic environments and low-bandwidth communication conditions.

E. Summary and Lessons Learned

Based on the comprehensive review of existing literature, several key observations are summarized as follows:

- *Centralized frameworks limit scalability and resilience:* Centralized frameworks can achieve good solutions, but

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

they are computationally expensive and vulnerable to single-point failures, whereas distributed frameworks offer greater scalability and robustness at the expense of optimality. Hence, designing lightweight distributed mechanisms with near-optimal performance remains a practical necessity for real-world SAR operations.

- *Balancing efficiency and quality:* Performance impact–based approaches enhance allocation quality through deeper optimization but suffer from slower convergence. Balancing algorithmic efficiency and allocation quality is crucial for time-critical missions.
- *Limited handling of uncertainty:* Real-world environments involve uncertain task information, constrained communication, and strict temporal and resource limits. Most existing methods address these factors separately, but effective coordination requires their integrated consideration to ensure mission success.
- *Impact of temporal and communication constraints:* Auction-based methods enable decentralized, fast decisions but rely on frequent communication. As team size and mission complexity grow, delays and bandwidth limits hinder coordination, making communication-efficient yet convergent task allocation a key challenge.

Building upon these observations, we propose a prediction-based dual-stage distributed task allocation method that integrates bid prediction, deadline-aware cost modeling, and post-allocation local search to enhance both convergence efficiency and allocation performance.

III. Problem Formulation

A. General Scenario Description

Consider a fleet of heterogeneous robots capable of performing both search and delivery tasks, deployed over the affected area. These robots continuously patrol the disaster zone to detect potential survivors. Upon detection, the detecting robot transmits critical survivor information (e.g., location and service time windows) to neighboring robots. Each service task is subject to specific deadline constraints and material requirements. Survivors may require one of two services: medicine or food delivery.

Once survivor information is shared, the robot swarm enters a decision-making process to allocate limited resources effectively. The objective is to complete supply delivery tasks under limited resource constraints and within each survivor's service window. Based on their individual capabilities and status, each robot first performs local computations to generate an initial task list. Then, through a consensus-based coordination mechanism, the robot swarm refines a joint rescue plan. Finally, the robot team delivers medicine or food packages to survivors according to the finalized plan. The overall flow of the collaborative SAR scenario is illustrated in Fig. 1.

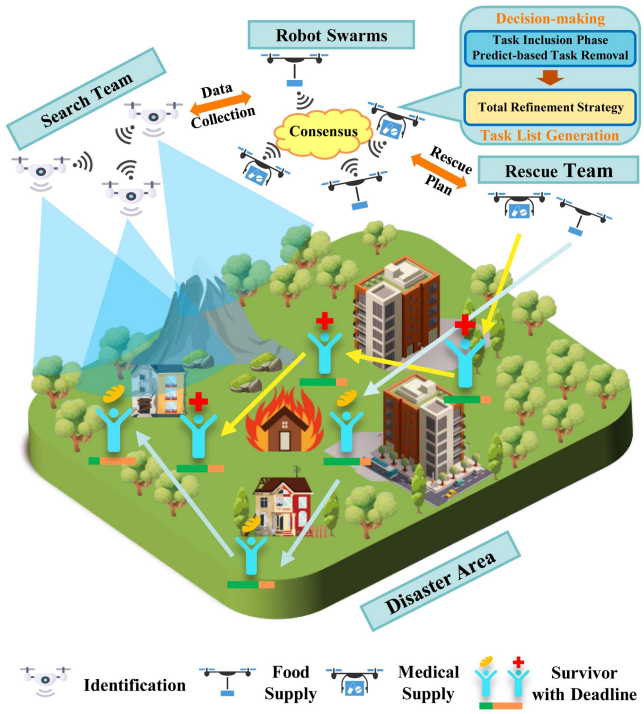


Fig. 1. Schematic diagram of a collaborative search and rescue scenario.

B. Mathematical Formulation

Consider a fleet of n autonomous robots, denoted as $\mathcal{V} = \{\upsilon_1, \upsilon_2, \ldots, \upsilon_n\}$, and a set of m survivors, represented as $\mathcal{T} = \{T_1, T_2, \ldots, T_m\}$. The primary objective is to provide supply delivery service to these survivors, who may require either medicine or food. To accomplish this, suitable robots must be dispatched to the survivors' locations. Each robot executes its assigned tasks in a predetermined order specified in its task list, and the entire rescue team is not mandated to return to the original departure point.

Tasks are categorized based on their specific requirements, and each is exclusively assignable to robots equipped to perform them. The assignment relationship is articulated through the compatibility matrix $\mathcal{H}$, where each element $h_{i,j} \in [0,1]$ denotes whether robot υ_i can execute task T_j ($h_{i,j} = 1$ if the robot can execute the task, otherwise 0. Each task T_j is associated with a deadline η_j and a predefined execution duration τ_j. To ensure successful task completion, a robot must commence its task before the deadline η_j.

Additionally, the operation time of robot υ_i is constrained by its fuel capacity, with a maximum available operational duration of f_i. Each robot is also limited in assignment to a maximum of $\mathcal{L}_i$ tasks, reflecting its transport capacity constraints. The task allocation challenge aims to maximize the number of survivors successfully served, subject to constraints on robot suitability, task deadlines, fuel limitations, and task compatibility. Within this context, the optimization problem is defined as follows:

$$J = \max\left\{\sum_{i=1}^{n} |\alpha_i|\right\} \tag{1}$$

$$\text{s.t.} \begin{cases} |\alpha_i| \le \mathcal{L}_i, & \forall i \in \mathcal{V}, \\ t_{i\kappa}(\alpha_i) < \eta_{a_{ik}}, & \forall \kappa \le |\alpha_i|, i \in \mathcal{V}, \\ t_{ik}(\alpha_i) < f_i, & \forall i \in \mathcal{V}, \\ \alpha_i \cap \alpha_j = \emptyset, & \forall (i,j) \in \mathcal{V}^2, i \ne j, \\ h_{i,j} \in [0,1], & \forall j \in \alpha_i, i \in \mathcal{V}. \end{cases} \tag{2}$$

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

TABLE II
SYMBOL DEFINITIONS

Notations	Definitions
$\mathcal{V}$	Set of rescue robots
$\mathcal{T}$	Set of rescue tasks
η_j	Deadline of task T_j
τ_j	Execution time of task T_j
α_i	Task list of robot v_i
$\mathcal{L}_i$	Maximum number of tasks for robot v_i
$\|\cdot\|$	Number of elements in a list
f_i	Maximum operating time of robot v_i
$d_{j',j}$	Distance between two adjacent tasks in task list
δ_i	Winner list stored in robot v_i
ξ_i	Global significance list of robot v_i
ξ_i^*	Marginal significance list of robot v_i
$C_i(\alpha_i)$	Total cost for executing task list α_i by robot v_i
$\omega_{ij}^{\oplus}(\alpha_i)$	Inclusion Performance Impact (IPI) of task T_j on robot v_i
$\omega_{ij}^{\ominus}(\alpha_i)$	Removal Performance Impact (RPI) of task T_j on robot v_i
$\alpha_i \ominus T_j$	Removal of task T_j from task list α_i
$\alpha_i \oplus_\ell T_j$	Insertion of task T_j at ℓ-th position in list α_i

where α_i represents the task list allocated to robot i, ensuring that the length $|\alpha_i|$ does not exceed its maximum task capacity $\mathcal{L}_i$. Robots execute tasks sequentially as per the order in α_i, where $t_{i\kappa}(\alpha_i)$ denotes the arrival time at the κ-th task $T_{\alpha_{i\kappa}}$, and $\eta_{\alpha_{i\kappa}}$ is its deadline. Furthermore, there should be no task conflicts between lists α_i and α_j assigned to different robots i and j. For the reader's convenience, the primary symbols used throughout are summarized in Table II.

IV. PREDICTION-BASED DUAL-PHASE TASK ASSIGNMENT ALGORITHMS

This section presents a prediction-based dual-stage distributed task allocation method specifically designed to optimize the objective function defined in (1). The method initiates with the generation of an initial allocation solution through an enhanced performance impact algorithm framework that integrates a task prediction mechanism. Following this, a local search mechanism is employed to iteratively refine the initial solution, thereby yielding a more optimal and higher-quality allocation.

A. Cost Function

In scenarios where a team of robots performs a series of tasks, the activity time of each robot is decomposable into two fundamental components: execution time and travel time. Execution time denotes the cumulative duration necessary for the completion of all tasks assigned within a robot's task list, whereas travel time encompasses the period required to transit from the robot's initial position to the task site and between successive tasks. To optimize the efficiency of task allocation, it is crucial to judiciously sequence task execution. Given that execution times are constant, reducing travel time between adjacent tasks allows for the completion of a greater number of tasks within the robot's activity time. Grounded in this principle, the cost associated with a task is quantified by integrating both travel time and deadline cost.

The travel time cost $c_{i\kappa}^{(1)}(\alpha_i)$ of the κth task in the task list α_i is defined as follows

$$c_{i\kappa}^{(1)}(\alpha_i) = \begin{cases} d_{ij}/u_i, & \kappa = 1 \\ d_{j',j}/u_i, & \kappa > 1 \end{cases}, \tag{3}$$

where u_i represents the travel speed of robot i.

When $\kappa = 1$, d_{ij} represents the distance from the initial position of the robot to the first task. When $\kappa > 1, d_{j',j}$ represents the distance of two adjacent tasks (from the previous task $T_{j'}$ to T_j). $T_{j'}$ is the $\kappa - 1$th task in the task list α_i, and T_j is the κ-th task (e.g. $T_j = T_{\alpha_{i\kappa}}$).

The deadline cost $c_{i\kappa}^{(2)}(\alpha_i)$ is defined as follows

$$c_{i\kappa}^{(2)}(\alpha_i) = \eta_{\alpha_{i\kappa}} - t_{i\kappa}(\alpha_i), \tag{4}$$

with $\eta_{\alpha_{i\kappa}}$ being deadline of task $T_{\alpha_{i\kappa}}$. In the design of this cost function, tasks with earlier deadlines are prioritized for allocation due to their reduced deadline costs, which are calculated based on the temporal proximity of a task's deadline to its arrival time. As a result, tasks with later deadlines are deferred, thereby creating strategic gaps within the schedule that facilitate the insertion of additional tasks.

According to the above statement, the total cost for robot i to execute all its allocated tasks is defined as:

$$C_i(\alpha_i) = \begin{cases} \sum_{\kappa=1}^{|\alpha_i|} c_{i\kappa}^{(1)}(\alpha_i) \cdot \sum_{\kappa=1}^{|\alpha_i|} c_{i\kappa}^{(2)}(\alpha_i), & \text{if } \alpha_i \text{ is feasible,} \\ \infty, & \text{otherwise.} \end{cases} \tag{5}$$

In this context, if a task list is deemed infeasible, the total cost is assigned a value of infinity. This effectively prevents the selection of infeasible solutions during the optimization process.

B. Prediction-Based Initial Solution Generation

During the initial solution generation phase, akin to the PI algorithm, the core process involves iterative optimization through two primary steps: the task inclusion phase and the conflict resolution phase.

1) Task Inclusion Phase: At this stage, each robot sequentially adds tasks from the unassigned task set to its own task list until no more eligible tasks remain or the task list reaches its capacity. To effectively determine which tasks to include, it is essential to first introduce two key metrics that evaluate the significance of tasks: Inclusion Performance Impact (IPI) and Removal Performance Impact (RPI).

The IPI serves as a metric of a task's contribution to the total cost incurred by the robot when the task is incorporated into its task list. Specifically, the IPI associated with the insertion of task T_j into the current task list α_i of robot i is computed as follows:

$$\omega_{ij}^{\oplus}(\alpha_i) = \min_{\ell=1}^{|\alpha_i|+1} \{C_i(\alpha_i \oplus_\ell T_j) - C_i(\alpha_i)\}, \tag{6}$$

where $\alpha_i \oplus_l T_j$ represents the task list α_i after inserting task T_j into position ℓ.

The RPI is employed to assess the variation in total cost resulting from the removal of a task from the current task list

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

of a robot. Specifically, the RPI associated with the removal of task T_j from the task list α_i of robot i is defined as follows:

$$\omega_{ij}^{\ominus}(\alpha_i) = C_i(\alpha_i) - C_i(\alpha_i \ominus T_j), \tag{7}$$

where $\alpha_i \ominus T_j$ represents the current task list after removing task T_j.

Following the introduction, the task inclusion phase is elaborated upon, with its main process outlined in Algorithm 1.

Each robot i independently selects a candidate task set $\mathcal{V}_i^c$ from the pool of tasks that require execution. This candidate set includes tasks that satisfy the following conditions: First, the task requirements must align with the robot type, indicated by $h_{ij} = 1$. Second, the task should not already be assigned to any other robot, i.e., $j \notin \bigcup_{i=1}^{n} \alpha_i$ (see Line 1 in Algorithm 1). Subsequently, the robot iteratively adds tasks to its task list, constrained by its capacity ($|\alpha_i| \leq \mathcal{L}_i$). During this process, robot i maintains two lists: the global significance list $\xi_i = [\omega_{i,1}^{\ominus}, \omega_{i,2}^{\ominus}, \ldots, \omega_{i,m}^{\ominus}]$ and the marginal significance list $\xi_i^* = [\omega_{i,1}^{\oplus}, \omega_{i,2}^{\oplus}, \ldots, \omega_{i,m}^{\oplus}]$. The global significance list captures the final bids of all robots for every task following the negotiation phase of the preceding round of task allocation, while the marginal significance list documents the local bids of the robots for all tasks in the present round.

To minimize the total cost of the robot team, each robot preferentially selects the task with the largest significance difference, indicating the most substantial potential impact on reducing overall costs, and includes it in its task list.

We first define the significance difference as $\vartheta = \max_{\kappa=1}^{m}\{\xi_{i\kappa} - \xi_{i\kappa}^*\}$, which represents the difference between the marginal significance and the global significance of each task. This metric is used to evaluate whether adding a new task can further optimize the current allocation. When $\vartheta > 0$, the IPI for each task added to the current task list is computed using formula (6), as detailed in Line 2 of Algorithm 1, and the corresponding insertion position ℓ is recorded. At the commencement of the algorithm, the global significance list ξ_i is initialized to a very large value. robot i then inserts the task T_τ that results in the largest significance difference ϑ into the optimal position ρ in its task list α_i. Since task T_τ is now assigned to robot i, its winner list ($\delta_i = [\delta_{i1}, \delta_{i2}, \ldots, \delta_{im}]$) is updated, along with the start times and cost values of the tasks in the task list, as outlined in Lines 3-11 of Algorithm 1.

Conversely, if $\vartheta < 0$, this indicates that the IPIs of all tasks are greater than or equal to their current global significance, suggesting that none of the candidate tasks can further reduce the total cost or satisfy the task's time constraints. In such cases, the task inclusion process terminates. Furthermore, the inclusion of new tasks affects the significance of others within the task list. Therefore, upon the conclusion of the task inclusion phase in α_i, the global significance list ξ_i for robot i must be updated, as specified in Line 16 of Algorithm 1.

2) Consensus and Predict-Based Task Removal: During the task inclusion procedure, each robot independently constructs its task list, which can result in conflicts over the allocation of identical tasks among robots. To resolve these conflicts, the robot team must coordinate to reach a consensus. Specifically, in the consensus phase, robots communicate through a connected network, broadcasting three types of lists among neighboring robots: the winner list δ_i, which logs the task allocation results; the global significance list $\xi_{i\kappa}$, which records the bidding status of robots for task execution; and the timestamp list, which captures the latest communication times to facilitate information synchronization.

Algorithm 1: Task Inclusion Phase.

1: Find the candidate task set $\mathcal{V}_i^c$
2: **while** $|\alpha_i| \leq \mathcal{L}_i$ **do**
3: Compute the IPIs of candidate tasks
4: $\vartheta = \max_{\kappa=1}^{m}\{\xi_{i\kappa} - \xi_{i\kappa}^*\}$
5: **if** $\vartheta > 0$ **then**
6: $T_\tau = \arg\max_{\kappa=1}^{m}\{\xi_{i\kappa} - \xi_{i\kappa}^*\}$
7: $\rho = \arg\min_{\ell=1}^{|\alpha_i|+1}\{C_i(\alpha_i \oplus_\ell T_\tau) - C_i(\alpha_i)\}$
8: Insert task T_τ into α_i at the position ρ
9: Update robot winner list $\delta_{i\tau} = i$
10: Update significance $\xi_{i\tau} \leftarrow \xi_{i\tau}^*$
11: Update the start time and costs of the tasks in α_i
12: **else**
13: **break**
14: **end if**
15: **end while**
16: Update the RPIs of tasks in α_i

The consensus procedure adheres to the protocols outlined in the literature [77]. Each robot compares its global significance list with those from neighboring robots. For each task, the task with a higher RPI results in a more significant reduction in total cost upon removal, thus prioritizing allocation to the robot with the lower RPI. Throughout this process, the lists are continuously updated and disseminated across the network.

Upon the conclusion of the consensus phase, all robots achieve a consistent global significance list and winner list, indicating consensus. Each robot then enters the task removal process, eliminating tasks that are not designated as its own winners. Unlike the performance impact algorithm, which relies on the outdated global significance list from the last consensus round for task removal, our proposed method incorporates a task prediction mechanism. This conventional approach overlooks the fact that other robots are simultaneously performing task removal operations, potentially leading to the erroneous removal of tasks that should have been retained [78], [79], [80]. To address this, the prediction-based dual-stage distributed task allocation (PDTA) method leverages task list exchanges during communication to predict subsequent bids from other robots, offering a more up-to-date and reliable reference for the task removal phase.

The conflict resolution process is outlined in Algorithm 2. Initially, the conflicting task set d_i is identified, consisting of tasks from the task list α_i for which the corresponding winner in the winner list δ_i is not robot i (Line 1 in Algorithm 2). These tasks will be removed from robot i's task list. Similar to the task inclusion process, the decision to continue task removal is based on calculating the significance difference ϑ° (Line 2–3 in

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

Algorithm 2: Predict-based Task Removal Phase.

1: Find the candidate task list d_i intended to be removed
2: Flag = True
3: $\vartheta^{\circ} = \max_{\kappa=1}^{|d_i|}\{\xi_{i\kappa}[d_i] - \xi^*_{i\kappa}[d_i]\}$
4: **while** $\vartheta > 0$ **and** $d_i \neq \emptyset$ **do**
5: **if** Flag == False **then**
6: Flag $\leftarrow$ Significance predict phase $(\alpha_j, \hat{\xi}_j, \xi_{i,k^*})$
7: **else**
8: $T_{k^*} = \arg\max_{\kappa=1}^{|d_i|}\{\xi_{i\kappa}[d_i] - \xi^*_{i\kappa}[d_i]\}$
9: Remove T_{k^*} from α_i, $\alpha_i \leftarrow a_i \backslash T_{k^*}$
10: Remove T_{k^*} from d_i, $d_i \leftarrow d_i \backslash T_{k^*}$
11: Update ξ_i and cost of all remaining tasks in the path α_i
12: Flag = False
13: **end if**
14: **end while**
15: Update the winner robot list $\delta_i(d_i) \leftarrow \upsilon_i$

Algorithm 3: Significance Predict Phase.

1: **for** each agent $\upsilon_j \in V_i^c$ **do**
2: **if** $T_{k^*} \in \alpha_j$ **then**
3: Find $d_j \leftarrow \alpha_j[\delta_j(\alpha_j) \neq j]$
4: **if** $d_j \neq \emptyset$ **then**
5: $\vartheta^{\circ}_j = \max_{\kappa=1}^{|d_j|}\{\hat{\xi}_j[d_j] - \xi_j[d_i]\}$
6: **if** $\vartheta^{\circ}_j > 0$ **then**
7: $\hat{T}_{k^*} \leftarrow \arg\max_{\kappa=1}^{|d_j|}\{\hat{\xi}_j[d_j] - \xi_j[d_j]\}$
8: Remove $\hat{T}_{k^*}$ from α_j, update $\alpha_j \leftarrow \alpha_j \backslash \{\hat{T}_{k^*}\}$
9: Remove $\hat{T}_{k^*}$ from d_j, update $d_j \leftarrow d_j \backslash \{\hat{T}_{k^*}\}$
10: Update the RPI value $\hat{\xi}_j$ and cost of all remaining tasks in task list α_j
11: **end if**
12: **end if**
13: **if** $\xi_{i\kappa^*} < \hat{\xi}_{j\kappa^*}$ **then**
14: Flag = False
15: **else**
16: Flag = True
17: **break**
18: **end if**
19: **end if**
20: **end for**
21: **return** Flag

Algorithm 2). A greedy approach is employed to identify the task T_{k^*} that maximizes this significance difference. This task is then removed from both the task list α_i and the conflicting task set d_i. Finally, the cost values and the global significance list ξ_i of the remaining tasks in α_i are updated (Lines 7–13 in Algorithm 2).

The prediction phase follows (Line 6 in Algorithm 2). If the task T_{k^*} is also included in the task list α_j of another robot j, it becomes necessary to predict the impact of removing this task from robot j. Utilizing the same principles as those outlined in the earlier task removal phase, a new predicted significance list $\hat{\xi}_j$ is established (Lines 1—10 in Algorithm 3). For task T_{k^*}, if the condition $\xi_{i\kappa^*} < \hat{\xi}_{j\kappa^*}$ is satisfied, it suggests that removing the task from robot i does not lead to a reduction in the overall cost. Consequently, robot i retains the task and awaits the outcomes of the subsequent consensus round (Line 13–14 in Algorithm 3). Conversely, if $\xi_{i\kappa^*} > \hat{\xi}_{j\kappa^*}$, the task removal process persists, iteratively eliminating tasks until the conflicting set d_i is exhausted or $\vartheta < 0$, indicating that further task removals would not decrease the total cost (Line 15–18 in Algorithm 3).

Subsequently, the task inclusion phase is executed for each robot, and the aforementioned procedures are reiterated until no further changes occur in the consensus, task deletion, and task inclusion phases, marking the conclusion of the first phase of the algorithm (Line 7–15 in Algorithm 2).

C. Local Search Mechanism and Refinement Strategy

Despite the algorithm generating an initial solution that meets deadline constraints, some tasks may remain unassigned. Consequently, this section presents a rescheduling method designed to optimize the initial task allocation, enabling the robot team to undertake additional tasks while ensuring all constraint conditions are upheld.

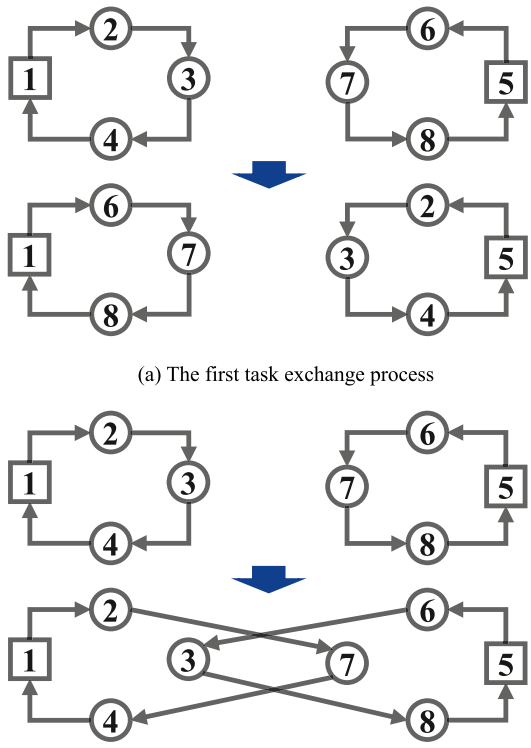
1) Local Search Mechanism: According to the designed cost function, reducing the total cost corresponds to increasing more available time intervals in a robot's task list, thereby facilitating the insertion of additional tasks. In scenarios where two robots possess identical service capabilities and where assigned tasks have equivalent service requirements, a local search mechanism is implemented to determine whether a lower total group cost can be attained. Consider two robots, i and j, with equal capabilities, and let their respective task lists be denoted as $\alpha_i = [\alpha_{i1}, \alpha_{i2}, \ldots, \alpha_{i\kappa}]$ and $\alpha_j = [\alpha_{j1}, \alpha_{j2}, \ldots, \alpha_{j\lambda}]$, where $\alpha_{i\kappa}$ and $\alpha_{j\lambda}$ represent the κ-th and λ-th tasks of robots i and j, respectively. The first approach involves exchanging the entire task lists of the two robots, swapping α_i with α_j, to explore task allocation across robots. The second approach entails local exchanges within task lists by swapping task α_{ik} in α_i with task $\alpha_{j\lambda}$ in α_j. Fig. 2 provides a schematic diagram of these task exchange methodologies. By integrating these strategies, we can adjust robot task allocations without altering the total number of tasks assigned, thereby perturbing the initial solution. This approach mitigates convergence to local optima and further explores the solution space. The main procedure is delineated in Algorithm 4.

2) Refinement Strategy: After the local search mechanism perturbs the initial solution, a task allocation result $\mathcal{D}$ with a reduced cost is obtained. Subsequently, a task inclusion procedure, analogous to the one outlined in Algorithm 1, attempts to insert tasks from the set of unassigned tasks $\mathcal{U}$ into the current solution.

The marginal significance list $\xi^*(\alpha_\upsilon, T)(\upsilon \in \mathcal{V}, T \in \mathcal{U})$ is calculated for all tasks in $\mathcal{U}$, with the highest marginal significance denoted as $\vartheta^+ = \max \xi^*(\alpha_\upsilon, T)$. This list quantifies the impact of inserting tasks, indicating how each candidate task's insertion affects the current robot's cost. The value ϑ^+ serves as a criterion for deciding task inclusion.

If $\vartheta^+ < M$, it implies the potential for including additional unassigned tasks. The tasks selected for addition are determined

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.



(a) The first task exchange process

(b) The second task exchange process

Fig. 2. Schematic diagram of two task exchange methods.

Algorithm 4: Local Search Mechanism.

```
while U ≠ ∅ do
  Complete list of exchange robots (2a)
  if Overall cost reduction then
    Update task lists
    break
  else
    Swap a list of robots for local tasks (2b)
    if Overall cost reduction then
      Update task lists
      break
    end if
  end if
end while
```

using the following formula:

$$(v^*, T^*, \rho^*) = \underset{v \in V, T \in \mathcal{U}}{\arg\min}\, \xi^*(\alpha_v, T), \tag{8}$$

where the triple (v^*, T^*, ρ^*) represents the combination in the task list α_v of robot v^*, the inclusion performance impact corresponding to the insertion position ρ^* of task T^* is minimized.

Subsequently, as shown in line 9 of Algorithm 5, the task list α_{v^*} and the unassigned task set $\mathcal{U}$ are updated according to the following formula:

$$\alpha_{v^*} = \alpha_{v^*} \oplus_{\rho^*} T^*, \mathcal{U} = \mathcal{U} \setminus \{T^*\}, \tag{9}$$

where $\alpha_{v^*} \oplus_{\rho^*} T^*$ denotes the insertion of task T^* into position ρ^* in the task list α_{v^*} of robot v^*, and $\mathcal{U} = \mathcal{U} \setminus \{T^*\}$ indicates the removal of task T^* from the unassigned task set $\mathcal{U}$.

Algorithm 5: The Total Refinement Strategy With Local Search Mechanism.

```
1:  done ← 1
2:  while done == 1 do
3:    if U = ∅ then
4:      break
5:    end if
6:    D ← Local search Mechanism(S_0)
7:    (ϑ+, v*, T*, ρ*) ← Task inclusion (U, D)
8:    if ϑ+ < M then
9:      (U, D) ← Update task list(D, v*, T*, ρ*, U)
10:   else
11:     Compute (ϑ', v1*, T1*, v2*, T2*, ρ2*) from (11)
12:     if ϑ' > 0 then
13:       α_{v1*} = α_{v1*} ⊖_{ℓ1*} T1*, U = U ∪ {T1*}
14:       α_{v2*} = α_{v2*} ⊕_{ℓ2*} T2*, U = U \ {T2*}
15:       done ← 1
16:     else
17:       done ← 0
18:     end if
19:   end if
20: end while
```

If $\vartheta^+ \geq M$, it indicates that no new tasks can be added through the local search mechanism. In this case, an assigned task with the highest RPI value is removed, and an unassigned task is added to further refine the solution space. This process aims to reduce the overall cost, after which local search optimization is reapplied.

Calculate the maximum significance difference between robots ϑ' as follows:

$$\vartheta' = \max \xi(\alpha_{v_1}, T_1) - \xi^*(\alpha_{v_2}, T_2). \tag{10}$$

If $\vartheta' > 0$, it can be inferred that the task allocation result can still be further optimized. Then, task T_1^* can be removed from the task list of robot v_1^*, and task T_2^* can be inserted at position ρ_2^* in the task list of robot v_2^*, creating a gap for the insertion of subsequent tasks (Lines 8– 18 in Algorithm 5). The robot-task pair can be computed using the following formula.

$$\begin{pmatrix} \vartheta', v_1^*, T_1^*, \\ v_2^*, T_2^*, \rho_2^* \end{pmatrix} = \underset{\substack{v_1, v_2 \in V, T_1 \in \alpha_{v_1} \\ T_2 \in \mathcal{U} \cup \{T_2\}}}{\arg\max} \{\xi(\alpha_{v_1}, T_1) - \xi^*(\alpha_{v_2}, T_2)\}. \tag{11}$$

Fig. 3 illustrates the overall workflow of the proposed PDTA algorithm. The algorithm begins with task broadcasting and operates in a distributed structure. During this phase, all tasks are transmitted to the robot team.

Each robot then enters the task inclusion phase, during which it independently generates a preliminary task list based on its capabilities, availability, and task compatibility: for example, robot 1 selects tasks 1, 2, 3, and 4, while robot 2 selects tasks 1, 2, 7, and 8.

Next, the robots enter a consensus phase, exchanging their individual task lists and task significance values (quantified based on urgency, deadlines, and location) with neighboring robots

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

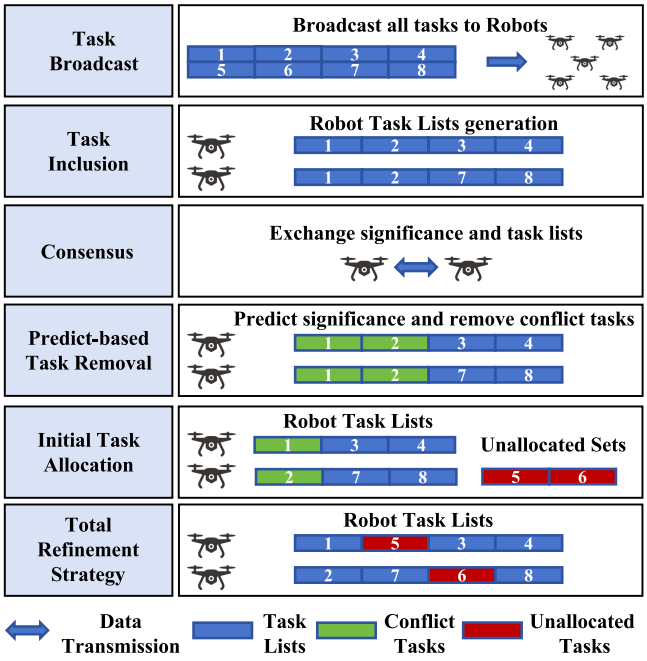


Fig. 3. Overview of the proposed PDTA algorithm workflow.

over the communication network. This process facilitates the identification of potential task conflicts. Following consensus, the robots proceed to a prediction-based task removal phase, in which each robot anticipates the bidding behavior of others and removes conflicting tasks accordingly. After several iterations of exchanging task lists and estimating task significance, conflicting tasks 1 and 2 are identified and removed, resulting in an initial task allocation: robot 1 is assigned tasks 1, 3, and 4, while robot 2 is assigned tasks 2, 7, and 8.

Subsequently, in the total refinement phase, each robot applies local optimization to insert unassigned tasks into its list. The remaining unallocated tasks 5 and 6 are inserted, and the robot's path list is updated in task order and execution times to produce the final service plan.

D. Complexity Analysis

Considering the distributed nature of the problem, it is necessary to analyze the computational complexity of the program executed by each robot to assess the feasibility of the proposed algorithm. As the proposed method adopts a two-stage distributed task allocation framework, we conduct complexity analysis separately for each stage.

Prediction-based initial solution generation: First, for the task inclusion phase in this phase, the complexity analysis follows the same approach as the PI algorithm. Its time complexity is given by $O((m_i - |a_i|)|a_i|^2 \vartheta_y \sigma)$, where $|a_i|$ denotes the cardinality of the task list assigned to robot i, m_i is the task-handling capacity, σ is the complexity of computing the time cost of a task, and ϑ_y represents the number of candidate tasks not yet included in the task list that satisfy the compatibility constraints.

In the prediction-based task removal phase, it is assumed that robot i identifies n_j other robots within its communication range with which it has task removal conflicts, and each of these robots has θ_{r_j} tasks to remove. The time complexity of this phase for robot i is $(|a_j|\theta_{r_j} - \frac{\theta_{r_j}(\theta_{r_j}+1)}{2} + (|a_j|-1))\sigma n_j$. Subsequently, robot i compares the predicted results with the significance of previously removed tasks and determines the number of tasks to be removed as θ_x, which are then removed. The corresponding complexity of this step is $((|a_i|-1)\theta_x - \frac{\theta_x(\theta_x+1)}{2} + (|a_i|-2))\sigma$. Combining the above, the total time complexity of the prediction-based initial solution generation phase is

$$O\left((m_i - |a_i|)|a_i|^2 \vartheta_y \sigma + |a_i|\theta_x \sigma + |a_j|\theta_{r_j} n_j \sigma\right). \quad (12)$$

Local search mechanism and refinement strategy: Assume that there are N robots in total, each robot has a maximum task capacity of m, with at most $\mathcal{L}$ tasks currently assigned (i.e., $|a_i| \leq \mathcal{L}$), and the process iterates for at most R rounds. This phase is divided into the following three stages for analysis:

In the local search mechanism, the algorithm iterates through all robot pairs with the same service capability within a heterogeneous swarm, attempting task exchanges only when both robots' capabilities and the service requirements of their assigned tasks match. The complexity of this stage is $N^2(\mathcal{L}^3+1)\sigma$. Then, the algorithm attempts to insert unallocated remaining tasks into the current feasible solution. Assuming the number of unassigned tasks is θ_z, the complexity of this insertion stage is $(m-\mathcal{L})\mathcal{L}^2\theta_z\sigma$. In the refinement stage, the algorithm attempts to replace existing tasks in the current solution with unassigned tasks to reduce overall cost and further improve task allocation. For all robot pair combinations and unassigned tasks, the complexity of refinement strategy is $(\theta_z+1)N\mathcal{L}^2\sigma$. Therefore, the total time complexity after up to R rounds of iteration is $O(RN^2\mathcal{L}^3\sigma + R(m-\mathcal{L})\mathcal{L}^2\theta_z\sigma + R(\theta_z+1)N\mathcal{L}^2\sigma)$.

In general, the computational burden on each robot is primarily determined by the second phase. This process exhibits polynomial time complexity. It is worth noting that the above analysis considers the worst-case complexity. In fact, simulation results demonstrate that the algorithm is capable of finding satisfactory solutions within a shorter computation time.

Remark 1: By introducing a task bid prediction mechanism and a local search refinement strategy, the proposed PDTA algorithm exhibits substantial advantages in increasing the number of allocated tasks and enhancing overall execution efficiency. Despite its strong performance, the algorithm has certain limitations. Specifically, the integration of the local search mechanism and refinement strategy increases computational complexity, which may not be ideal for robot teams with relatively limited computing resources.

Moreover, the current version of the algorithm assumes a static task set and environment. However, in real-world applications, dynamic factors such as robot failures, task updates, or priority changes often require real-time reallocation. This limitation affects the responsiveness of the algorithm in dynamic environments. Future work will focus on extending the algorithm to handle dynamic task allocation and improve its adaptability to real-time changes in multi-robot systems.

Remark 2: This remark is provided to elaborate on the basis for setting task deadlines in the scenarios. In post-disaster SAR

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

scenarios, tasks often differ significantly in urgency depending on the type of service required by the survivor. For example, survivors requiring medical supplies need a rapid response, and their service windows are relatively short to ensure timely intervention. In contrast, logistical tasks, such as delivering food or supplies to survivors in stable condition, generally have longer service windows.

E. Convergence and Optimality Analysis

The proposed PDTA algorithm is built upon the iterative framework of the PI algorithm, continuously optimizing the global objective through multiple iterations in the task allocation process. In each iteration, robots evaluate the significance of tasks based on current information and dynamically adjust their inclusion and removed lists, gradually driving the system toward convergence. The algorithm terminates when no robot can further reduce the overall cost or make effective modifications to its task list under the current state. It is worth noting that prior studies have shown that PI-based frameworks may fall into infinite task exchange loops in certain scenarios, preventing convergence within a finite time. To address this issue, the PDTA algorithm employs a control mechanism similar to those used in other PI-based algorithms: once the number of times a task has been removed from a robot's list reaches a predefined threshold, the robot will no longer attempt to re-include that task. This mechanism effectively avoid repetitive exchange cycles and ensures strong convergence of the algorithm.

To validate the convergence and performance of PDTA, we use the exact optimal solutions obtained from the corresponding mixed-integer programming (MIP) models solved by Gurobi as performance benchmarks. The goal is to maximize the number of successful rescue missions. A total of 27 comparative experiments were conducted under varying combinations of robot numbers (4-6), task quantities (16-36), and robot capacities (4-6).

As shown in Table III, PDTA consistently achieved near-optimal results across all test instances. Although Gurobi guarantees global optimality of solution, the problem is inherently NP-hard, resulting in rapidly increasing computational complexity as the number of robots and tasks grows. For instances with up to 6 robots and 36 tasks, the solver already requires considerable computational time. The average relative gap of our proposed PDTA algorithm from optimal solution of Gurobi is only 5.23%, with the minimum gap as low as 0.30% and the maximum not exceeding 8.10%. In small-scale instances (e.g. Robots=4, Tasks=16), PDTA nearly reaches optimality. Despite a slight increase in the solution gap under higher task densities or larger robot capacities, the algorithm still maintains a high level of solution quality.

Among existing distributed task allocation algorithms, the well-known consensus-based bundle algorithm, under the assumption of diminishing marginal gain (DMG), asymptotically converges to the solution of the centralized sequential greedy algorithm. However, even under the DMG condition, CBBA cannot guarantee global optimality, and its theoretical optimality bound is limited to 50%. Other baseline methods based on performance impact algorithm, such as PIsoftmax, PImaxAss, EEPI and PIDopt, do not guarantee optimality but aim to maximize task success rates while ensuring feasible solutions. In contrast, as a distributed task allocation method, PDTA algorithm demonstrates high-quality, convergent, and near-optimal performance across varying problem scales and densities.

TABLE III
PDTA VS GUROBI: OPTIMALITY ACROSS TASK DENSITIES

Ratio	Robots	Tasks	Capacity	PDTA	Gurobi	Gap(%)
4	4	16	4	12.762	12.800	0.30
	4	16	5	12.762	12.800	0.30
	4	16	6	12.762	12.800	0.30
	5	20	4	15.655	16.350	4.25
	5	20	5	15.702	16.400	4.26
	5	20	6	15.702	16.400	4.26
	6	24	4	20.000	20.700	3.38
	6	24	5	19.988	20.750	3.67
	6	24	6	19.988	20.750	3.67
5	4	20	4	13.143	13.800	4.76
	4	20	5	13.238	14.050	5.78
	4	20	6	13.238	14.050	5.78
	5	25	4	16.583	17.400	4.70
	5	25	5	16.726	17.700	5.50
	5	25	6	16.726	17.700	5.50
	6	30	4	20.143	21.650	6.96
	6	30	5	20.357	22.100	7.89
	6	30	6	20.357	22.100	7.89
6	4	24	4	13.191	14.000	5.78
	4	24	5	13.286	14.300	7.09
	4	24	6	13.286	14.300	7.09
	5	30	4	16.548	17.750	6.78
	5	30	5	16.726	18.200	8.10
	5	30	6	16.726	18.200	8.10
	6	36	4	20.762	21.900	5.20
	6	36	5	21.083	22.700	7.12
	6	36	6	21.083	22.700	7.12

V. SIMULATIONS

This section presents the results of numerical simulations conducted to assess the efficacy and performance of the proposed distributed task allocation algorithm. Section V-A outlines the scenario setup and describes the SAR simulation environment. Section V-B provides comprehensive simulation outcomes, comparing the proposed algorithm with several established distributed task allocation algorithms under varying initial conditions, robot counts, and task numbers. All simulations were performed on a single platform equipped with an Intel Core i7-12700F processor running at 2.10 GHz and 16 GB of RAM.

A. Simulation Setup

In post-disaster scenarios, it is crucial to rapidly deploy multiple types of rescue teams to deliver emergency support to survivors, especially under constrained communication conditions. The survivors' needs are classified into two categories: medical supplies and food. Accordingly, the rescue teams are divided into two types, one transporting medicine and the other transporting food. To validate the effectiveness of the proposed PDTA method, standard SAR scenarios, as referenced in [78], [79], and [80] are used as benchmarks.

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
SCENARIO DESCRIPTION: MEDICINE AND FOOD DELIVERY

Parameter	Medicine Task	Food Task
Robot Speed	30 m/s	50 m/s
Fuel Limitation	[3500s, 4000s]	
Initial Position	[10km, 10km] (2D Plane)	
Task Execution Window (s)	300	350
Task Deadline (s)	[0, 4000]	
Task Location (km)	[10, 10, 1] (3D Space)	

In the numerical simulations, the scenario parameters are detailed in Table IV. Survivors are randomly distributed within a 3-D space measuring $10\,\text{km} \times 10\,\text{km} \times 1\,\text{km}$. All tasks are assigned equal priority, with time windows of 300 seconds for medicine delivery tasks and 350 seconds for food delivery tasks. The earliest start time for each rescue task is set to 0 seconds, and deadlines are randomly distributed between 0 and 4000 seconds. robots are randomly initialized within the corresponding $10\,\text{km} \times 10\,\text{km}$ 2-D ground space, operating at fixed speeds of 30 m/s and 50 m/s. Each robot has a fuel limit ranging from 3500 to 4000 seconds.

The proposed PDTA algorithm is a task scheduling method designed within the framework of distributed systems. In practical scenarios, robots cannot communicate directly if their separation distance exceeds the communication range. Consequently, the communication network topology becomes a critical parameter for evaluating the robustness of the algorithm. To address this, four topological structures—mesh, row, circular, and star—are employed as the communication framework during each iteration. These structures simulate actual network communication within robot groups and facilitate the assessment of the algorithm's performance under the limited perception ranges characteristic of robot groups. The topologies are illustrated in Fig. 4, and are randomly regenerated in each iteration, featuring bidirectional communication between nodes. Serial numbers are provided for reference purposes only.

A comparative analysis was conducted between the proposed PDTA algorithm and four leading distributed task allocation algorithms: CBBA, PI, PImaxAss, and PIDopt. The CBBA algorithm serves as a well-established benchmark for distributed task allocation problems. The PI algorithm, which focuses on minimizing the average task completion time, exemplifies optimization for temporal performance, while PImaxAss algorithm represents the current optimal solution for maximizing the number of allocated tasks. PIDopt algorithm is the latest work based on the PI framework. Therefore, this comparison offers a comprehensive evaluation of the proposed algorithm's performance.

Simulations were conducted under each of the four topological structures. To mitigate the effects of randomness, each algorithm underwent 50 Monte Carlo simulations with randomly generated task scenarios, using the same random seed across runs to ensure fairness. The tests encompassed various robot-to-task ratios and multi-task allocation scenarios. The primary performance metrics included the average number of task allocations and the average number of algorithm iterations. Notably,

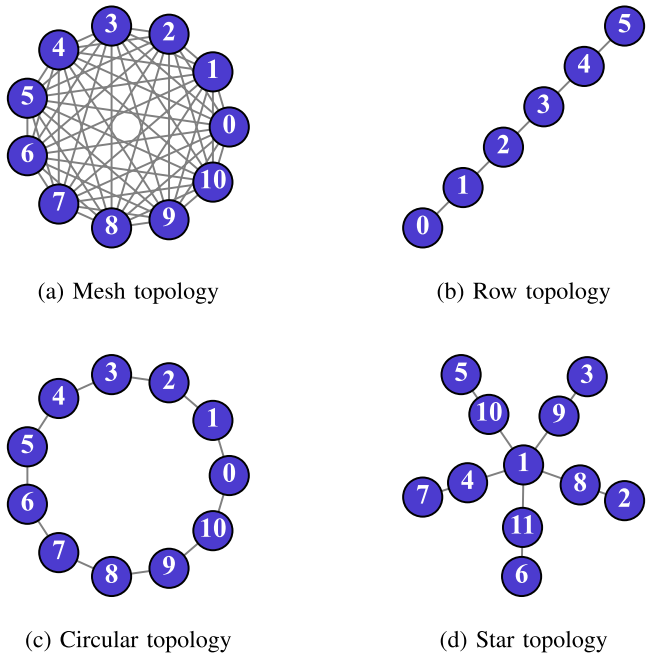


Fig. 4. Four different communication network topologies.

in the simulations, to guarantee the algorithm's convergence, the maximum number of times each robot can delete the same task is limited to five. Additionally, due to the random initialization of tasks and robot positions, some tasks may not be completed by any robot before their deadlines, rendering them infeasible. Prior to executing the algorithm to determine task allocation results, all task information is disclosed to the robot team. Ultimately, each robot attains a globally consistent allocation outcome through local computations and consensus achieved via mutual communication.

B. Simulation Results

Initially, the five algorithms were evaluated under different robot-to-task ratios. The number of robots was set to 8, 12, 14, 16 and 18, corresponding to ratios of 5, 6, 7, and 8, respectively. Half of the robots were designated for medicine delivery, while the other half focused on food delivery. Table VI displays the simulation results for each algorithm across these multi-task ratios, showcasing the average number of task assignments and average number of iterations. The maximum values for task assignments and the minimum number of iterations for each scenario are highlighted in bold.

The simulation results across the four robot-to-task ratios indicate that the PDTA algorithm outperforms the other five algorithms, achieving a higher average number of task assignments and requiring fewer iterations, as shown in Table VI. It is notable that when the number of tasks is small, some tasks remain incomplete owing to early deadlines (with certain robots unable to reach the tasks before their deadlines). However, the remaining tasks are completed with a high success rate by our algorithm, as well as by the PI and PImaxAss algorithms, whereas the CBBA algorithm exhibits a comparatively lower completion rate. Figs. 6–9 depict the simulation outcomes of the different

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

TABLE V
TASK INFORMATION SUMMARY AND SCHEDULING RESULTS

Task ID	1	2	3	4	5	6	7	8	9	10	11	12	13	14
Duration	300	300	300	300	300	300	300	350	350	350	350	350	350	350
Type	1	1	1	1	1	1	1	2	2	2	2	2	2	2
Deadline	1397.87	2624.33	2186.80	1914.35	1022.15	2451.94	2668.43	633.86	2508.23	2103.88	2085.47	1070.57	1006.30	1546.09

Algorithm		Scheduling Result							Completion Rate	Completed Tasks	Iterations
PDTA	R1 TaskList	1	6	5	4	3	2	7	**92.9%**	**13/14**	4
	Arrival Time	104.7	582.6	1009.2	1381.4	1806.7	2225.9	2635.4			
	R2 TaskList	8	13	12	14	11	9	–			
	Arrival Time	153.3	564.1	981.6	1516.2	1964.3	2334.0	–			
Gurobi	R1 TaskList	1	5	6	4	3	2	7	**92.9%**	**13/14**	—
	Arrival Time	104.7	539.7	966.3	1369.1	1794.4	2213.6	2623.2			
	R2 TaskList	8	12	13	11	10	9	–			
	Arrival Time	153.3	513.0	930.5	1319.2	1726.0	2113.4	–			
PI	R1 TaskList	6	4	5	1	7	2	–	71.4%	10/14	4
	Arrival Time	80.7	483.5	855.7	1290.7	1871.9	2281.5	–			
	R2 TaskList	11	9	10	14	–	–	–			
	Arrival Time	79.0	448.6	836.1	1253.9	–	–	–			
PImaxAss	R1 TaskList	6	4	5	1	7	2	–	71.4%	10/14	7
	Arrival Time	80.7	483.5	855.7	1290.7	1871.9	2281.5	–			
	R2 TaskList	11	9	10	14	–	–	–			
	Arrival Time	79.0	448.6	836.1	1253.9	–	–	–			
PIDopt	R1 TaskList	6	4	5	7	2	3	–	71.4%	10/14	7
	Arrival Time	80.7	483.5	855.7	1290.7	1871.9	2281.5	–			
	R2 TaskList	11	9	10	14	–	–	–			
	Arrival Time	79.0	448.6	836.1	1253.9	–	–	–			
CBBA	R1 TaskList	7	2	6	3	–	–	–	57.1%	8/14	4
	Arrival Time	265.9	675.5	1176.8	1607.2	–	–	–			
	R2 TaskList	9	10	11	14	–	–	–			
	Arrival Time	92.0	479.5	886.2	1334.4	–	–	–			

TABLE VI
COMPARISON OF AVERAGE TASK ALLOCATION AND ITERATIONS FOR DIFFERENT RATIOS (EXTREMES HIGHLIGHTED IN BOLD)

Ratio	Robots: Tasks	Average Number of Task Allocation					Average Number of Iterations				
		PDTA	PIinit	PImaxAss	CBBA	PIDopt	PDTA	PI	PImaxAss	CBBA	PIDopt
Ratio=5	8:40	**38.93**	38.05	38.77	26.82	38.77	**32.45**	33.82	45.93	32.86	45.93
	12:60	**59.27**	58.39	59.16	40.82	59.16	**52.00**	69.73	86.68	53.55	86.68
	14:70	68.86	68.14	**68.95**	47.18	**68.95**	**62.91**	85.93	105.07	65.34	105.07
	16:80	**79.18**	78.34	79.09	55.55	79.09	**74.82**	105.25	126.61	81.52	126.61
	18:90	**89.18**	88.16	88.91	61.45	88.91	**90.45**	121.64	147.45	90.59	147.45
Ratio=6	8:48	**46.18**	44.43	45.73	30.18	45.73	**28.52**	34.43	48.07	35.39	48.07
	12:72	**70.57**	68.00	69.57	46.27	69.59	**51.18**	73.07	93.50	59.80	93.50
	14:84	**82.89**	80.39	82.41	52.82	82.41	**65.45**	84.41	108.61	71.66	108.61
	16:96	**94.64**	92.55	94.11	62.45	94.14	**73.07**	102.02	125.91	87.05	125.91
	18:108	**106.55**	104.18	106.18	69.18	106.18	**84.32**	138.30	167.11	102.36	167.11
Ratio=7	8:56	**52.95**	49.80	52.14	32.64	52.14	**27.82**	34.36	49.39	36.25	49.39
	12:84	**81.27**	77.05	80.09	50.18	80.09	**50.07**	60.00	83.09	62.41	83.09
	14:98	**96.02**	90.64	94.52	57.45	94.52	**54.89**	86.68	113.23	75.91	113.23
	16:112	**109.45**	104.41	108.18	67.82	108.18	**69.82**	102.86	133.27	92.52	133.27
	18:126	**123.93**	118.61	122.50	75.36	122.50	**76.14**	127.18	158.98	104.45	158.98
Ratio=8	8:64	**59.57**	54.11	58.34	35.18	58.39	**26.98**	31.70	48.95	36.36	48.95
	12:96	**91.20**	84.68	88.68	53.82	88.68	**46.93**	59.68	82.86	67.09	82.86
	14:112	**107.93**	100.02	105.00	63.64	105.00	**52.86**	80.32	108.05	82.14	108.05
	16:128	**123.25**	113.98	120.27	72.27	120.27	**62.27**	88.68	119.68	92.23	119.68
	18:144	**139.05**	129.30	136.02	80.64	136.07	**73.55**	126.16	163.50	114.11	163.50

algorithms for robot-to-task ratios of 7 and 8, respectively. As the number of tasks increases, the task allocation capability of some algorithms significantly declines due to constraints imposed by robot fuel limitations. The PDTA algorithm, in contrast, shows a marked improvement in task allocation performance, especially in scenarios with a high task ratio, which ensures a high task completion rate. In SAR scenarios, the number of completed tasks is the most critical performance indicator, as a lower number of successful rescues cannot satisfy actual operational requirements.

Fig. 5 illustrates the scheduling results under a row topology in a scenario with 2 robots and 14 tasks. Fig. 5 provides a visual representation of the scheduling results by projecting the three-dimensional spatial coordinates onto a two-dimensional

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

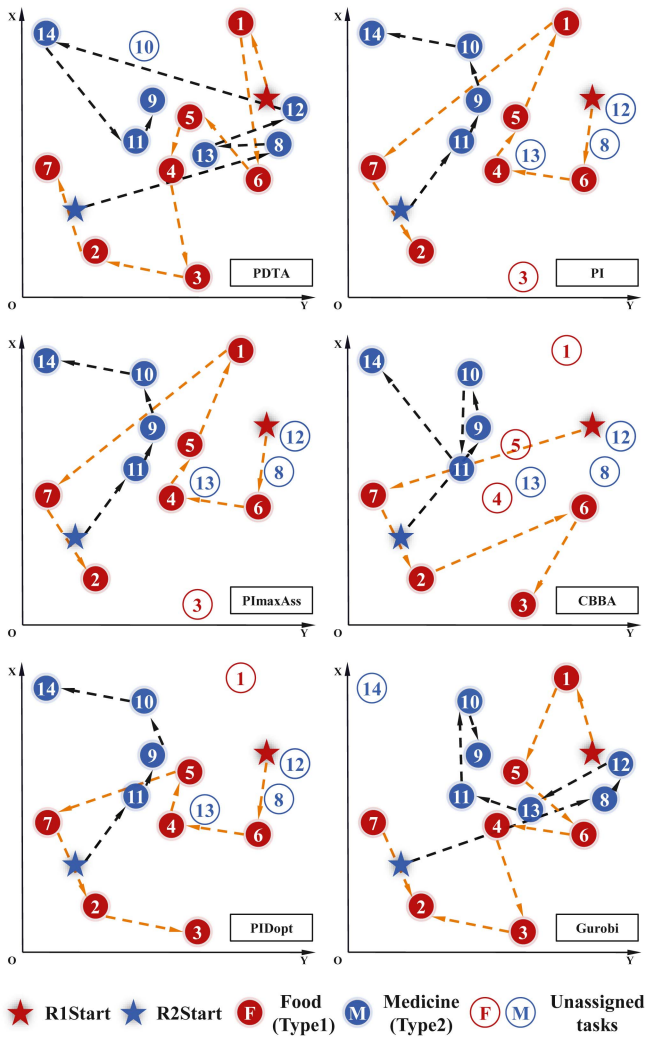


Fig. 5. Task scheduling diagrams for six algorithms: PDTA, PI, PImaxAss, CBBA, PIDopt, and Gurobi.

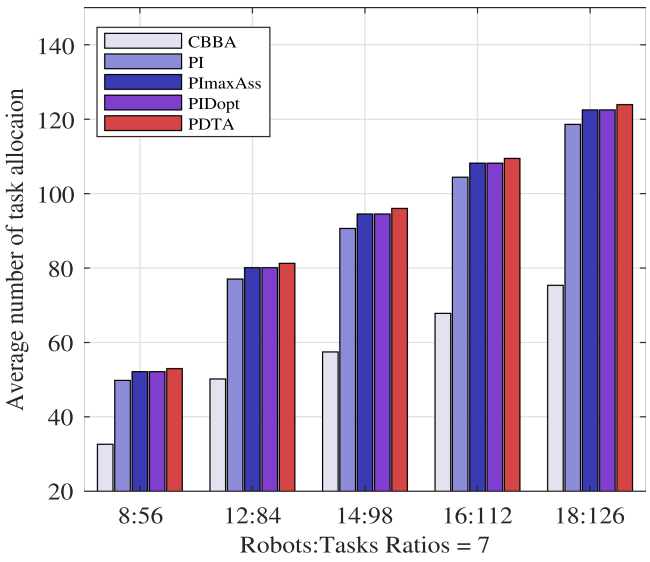


Fig. 6. Average number of task allocation under different algorithms (Robots:Tasks Ratios = 7).

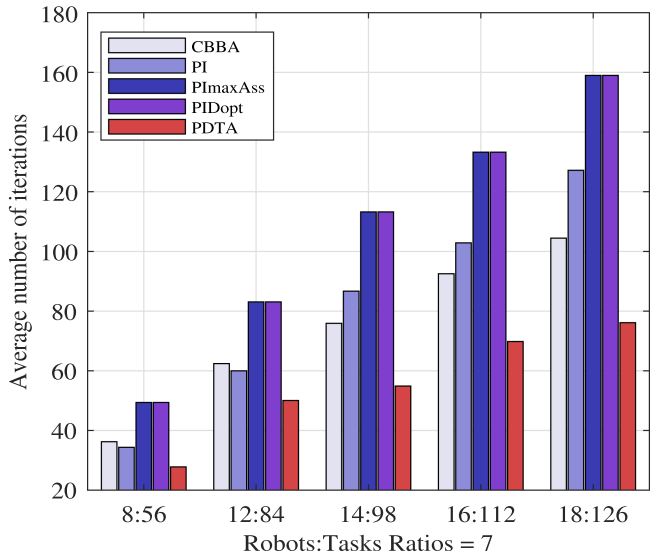


Fig. 7. Average number of iterations under different algorithms (Robots:Tasks Ratios = 7).

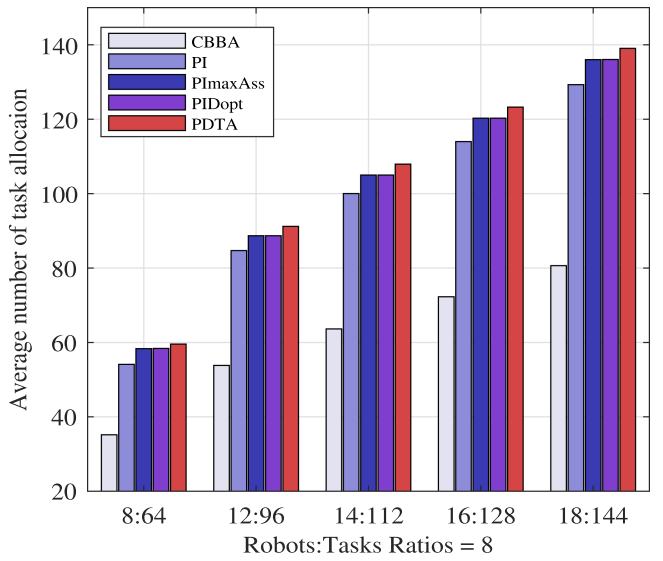


Fig. 8. Average number of task allocation under different algorithms (Robots:Tasks Ratios = 8).

plane, while Table V presents the complete numerical details of the task assignments. We use the exact solution of the mixed-integer program solved by Gurobi as the benchmark. The numbers of assigned tasks are 13, 13, 10, 10, 10, and 8, respectively. Among these allocation schemes, the PI, PImaxAss and PIDopt algorithms assign an identical number of tasks, our PDTA algorithm and gurobi allocates the highest number of tasks, and CBBA offers the fewest. Although our scheduling results differ slightly from the exact solution, this is expected since multiple feasible solutions may achieve the maximum completion rate. In contrast, exact methods are time-consuming, centralized, and demand significant computational resources. Both PI and PImaxAss use a scheduling strategy prioritizing tasks based on arrival times, leading to missed deadlines for

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

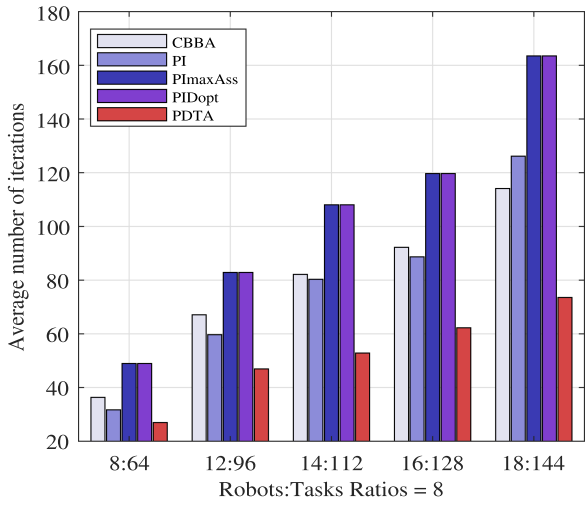


Fig. 9. Average number of iterations under different algorithms (Robots:Tasks Ratios = 8).

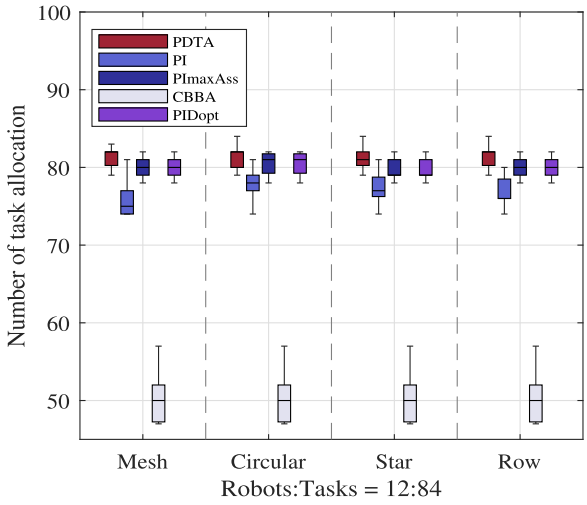


Fig. 10. Box plot of number of task allocation under four topologies (Robots:Tasks Ratios = 12:84).

tasks 3, 8, 12, and 13, and thus preventing their allocation. PIDopt, an algorithm based on PImaxAss, improves the scheduling order but does not increase the number of task assignments. It also completed 10 tasks; however, according to the scheduling result, tasks 1, 8, 12, and 13 were not executed. In contrast, our allocation approach strategically prioritizes tasks with later deadlines, creating additional task allocation opportunities and ultimately resulting in a higher number of task assignments. Consequently, our algorithm enhances the task completion rate through an efficient task allocation strategy, demonstrating its advantages in SAR operations.

In this simulation experiment, we selected four representative network topologies–Mesh, Row, Circular, and Star–to evaluate the impact of different communication networks on the performance of the task allocation algorithm. As illustrated in Fig. 4, in the Mesh topology, each agent has direct communication links with all other agents. In the Circular topology, each agent is connected only to its two adjacent agents. The Star topology forms a star-shaped communication network by randomly selecting a central node, while the Row topology arranges the agents in a linear sequence, where each agent communicates solely with its local neighbors. Figs. 11 and 13 depict the number of iterations under different topologies when the robot-task ratios are 12:84 and 12:96, respectively. In the Mesh topology, task information is rapidly propagated to all agents, enabling the highest negotiation efficiency and resulting in the fewest iterations. Conversely, the number of iterations in the Star and Circular topologies is slightly higher due to their restricted range of information propagation. The Row topology exhibits the largest number of iterations, primarily due to its extensive network diameter, which limits both the speed and reach of information dissemination, leading to an increase in the number of iterations. Although the final number of task allocations (Figs. 10 and 12) shows only slight variations across the different topologies, the differences in the number of iterations illustrate the significant impact of network topology on the performance of distributed algorithms.

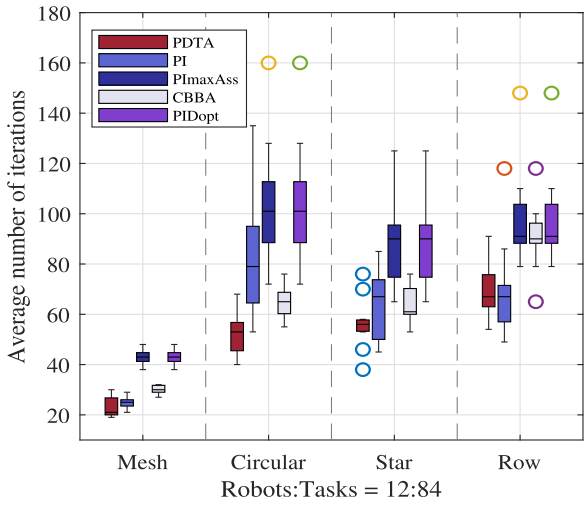


Fig. 11. Box plot of number of iterations under four topologies (Robots:Tasks Ratios = 12:84).

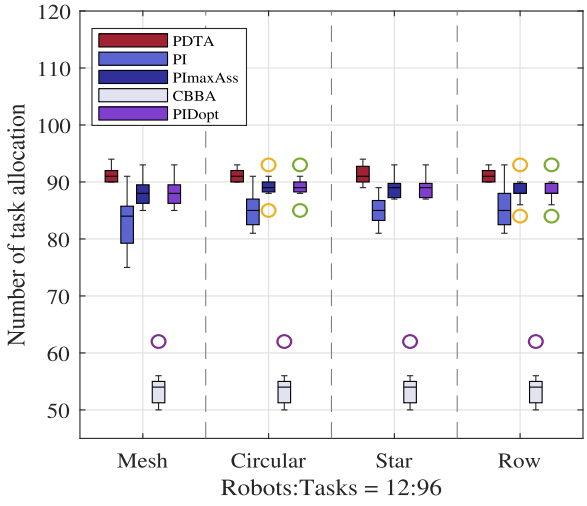


Fig. 12. Box plot of number of task allocation under four topologies (Robots:Tasks Ratios = 12:96).

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

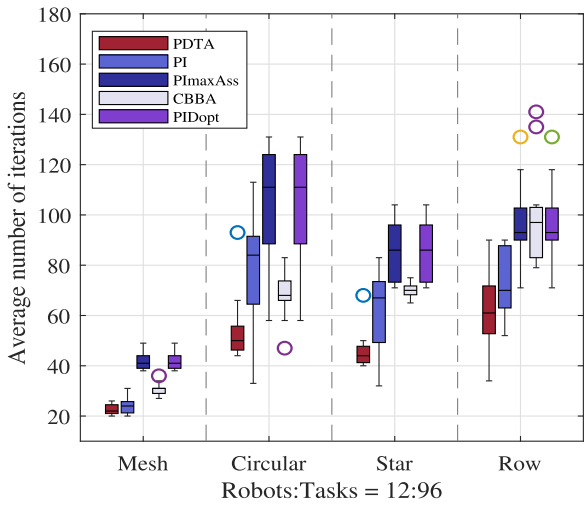


Fig. 13. Box plot of number of iterations under four topologies (Robots:Tasks Ratios = 12:96).

For a detailed discussion on the influence of network topology on distributed algorithm performance, please refer to [84].

Regarding the number of iterations, the PDTA algorithm in this study demonstrates a significant advantage in the task negotiation process. PIDopt follows a two-stage process, with the first stage identical to PImaxAss, resulting in the same number of iterations. Both PIDopt and PImaxAss, along with PI, optimize task allocation through multiple rounds of task negotiation and bid updates. In contrast, our method reduces unnecessary task removal operations during the task removal phase by predicting the bids of competing robots, effectively decreasing the number of iterations and accelerating consensus. As shown in Table VI, as the number of tasks increases, the CBBA algorithm's number of iterations rises significantly. This is because the CBBA algorithm necessitates the removal and reallocation of all subsequent tasks whenever a new task is introduced, leading to more iterations. Conversely, the PI-based algorithm only removes tasks with higher bids than those of competing robots, allowing it to complete task allocations in fewer iterations, particularly when the number of tasks is small or when the task count significantly exceeds the capabilities of the robot team. However, with a moderate number of tasks, some tasks are repeatedly selected and exchanged between multiple robots, resulting in a higher number of iterations required for the algorithm to converge.

VI. Conclusion

This paper addresses the problem of distributed multi-robot and multi-task allocation in SAR operations. We propose a prediction-based dual-stage task allocation (PDTA) algorithm to maximize the number of survivors rescued. During the task inclusion phase of initial solution generation, a deadline-sensitive cost function is designed to create time gaps in robot task lists, thereby improving the flexibility of task allocation. In the task removal phase, a task bid prediction mechanism is introduced to mitigate invalid bids, reducing the number of algorithm iterations and accelerating convergence. The second stage further refines the initial solution through a local search strategy, enhancing the overall solution quality. Simulation results demonstrate that the PDTA algorithm significantly outperforms existing methods in terms of task allocation efficiency and convergence speed, validating its effectiveness and reliability. Moreover, the proposed approach is broadly applicable to other scenarios that conform to the presented optimization model. Future work will focus on dynamic task allocation environments to further extend the algorithm's capabilities.

References

[1] M. Afrin, J. Jin, A. Rahman, A. Gasparri, Y.-C. Tian, and A. Kulkarni, "Robotic edge resource allocation for agricultural cyber-physical system," *IEEE Trans. Netw. Sci. Eng.*, vol. 9, no. 6, pp. 3979–3990, Nov./Dec. 2022.

[2] X. Qin et al., "Timeliness-oriented asynchronous task offloading in UAV-edge-computing systems," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 900–912, Jan./Feb. 2024.

[3] A. A. Al-habob, J. Lin, O. A. Dobre, and Y. Jing, "Min-max latency minimization for energy-constrained multi-UAV mobile edge computing," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 4577–4590, Sep./Oct. 2024.

[4] P. Qin, J. Li, J. Zhang, and Y. Fu, "Joint task allocation and trajectory optimization for multi-UAV collaborative air–ground edge computing," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 6, pp. 6231–6243, Nov./Dec. 2024.

[5] A. Gautam and S. Mohan, "A review of research in multi-robot systems," in *Proc. IEEE 7th Int. Conf. Ind. Inf. Syst.*, 2012, pp. 1–5.

[6] Z. Wang and J. Zhang, "A task allocation algorithm for a swarm of unmanned aerial vehicles based on bionic wolf pack method," *Knowl.-Based Syst.*, vol. 250, 2022, Art. no. 109072.

[7] Y. Wang, F. Zhu, and T. N. Dinh, "Distributed hybrid dynamic event-triggered bipartite consensus control for multi-agent systems against dos attacks," *IEEE Trans. Circuits Syst. I, Reg. Papers*, vol. 72, no. 12, pp. 8120–8131, Dec. 2025.

[8] Y. Wang and F. Zhu, "Distributed hybrid dynamic event-triggered consensus control for nonlinear multi-agent systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 10470–10 483, 2025.

[9] K. Jia, D. Yang, Y. Wang, T. Shui, and C. Liu, "Energy efficient and balanced task assignment strategy for multi-AAV patrol inspection system in mobile edge computing network," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 1, pp. 210–222, Jan./Feb. 2025.

[10] C. Xu and W. Song, "Intelligent task allocation for mobile crowdsensing with graph attention network and deep reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 2, pp. 1032–1048, Mar./Apr. 2023.

[11] F. Zitouni, R. Maamri, and S. Harous, "Towards a formal analysis of the multi-robot task allocation problem using set theory," *Bull. Elect. Eng. Informat.*, vol. 10, no. 2, pp. 1092–1104, 2021.

[12] P. Qin, M. Li, K. Wu, and Y. Fu, "Cost minimization resource allocation with service instance caching and task migration for UAV mobile edge computing," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 1738–1753, 2026.

[13] H. Chakraa, F. Guérin, E. Leclercq, and D. Lefebvre, "Optimization techniques for multi-robot task allocation problems: Review on the state-of-the-art," *Robot. Auton. Syst.*, vol. 168, 2023, Art. no. 104492.

[14] B. P. Gerkey and M. J. Mataric, "Sold!: Auction methods for multirobot coordination," *IEEE Trans. Robot. Automat.*, vol. 18, no. 5, pp. 758–768, Oct. 2002.

[15] S. Chopra, G. Notarstefano, M. Rice, and M. Egerstedt, "A distributed version of the Hungarian method for multirobot assignment," *IEEE Trans. Robot.*, vol. 33, no. 4, pp. 932–947, Aug. 2017.

[16] A. Jevtic, A. Gutiérrez, D. Andina, and M. Jamshidi, "Distributed bees algorithm for task allocation in swarm of robots," *IEEE Syst. J.*, vol. 6, no. 2, pp. 296–304, Jun. 2012.

[17] W. Dai, U. Rai, J. Chiun, Y. Cao, and G. Sartoretti, "Heterogeneous multi-robot task allocation and scheduling via reinforcement learning," *IEEE Robot. Automat. Lett.*, vol. 10, no. 3, pp. 2654–2661, Mar. 2025.

[18] D. Liu, L. Dou, R. Zhang, X. Zhang, and Q. Zong, "Multi-agent reinforcement learning-based coordinated dynamic task allocation for heterogenous UAVs," *IEEE Trans. Veh. Technol.*, vol. 72, no. 4, pp. 4372–4383, Apr. 2023.

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

[19] A. Khamis, A. Hussein, and A. Elmogy, "Multi-robot task allocation: A review of the state-of-the-art," in *Cooperative Robots and Sensor Networks*, A. Koubâa and J. R. M. de Dios Eds. Cham, Switzerland: Springer, 2015, pp. 31–51.

[20] B. Chen and H. H. Cheng, "A review of the applications of agent technology in traffic and transportation systems," *IEEE Trans. Intell. Transp. Syst.*, vol. 11, no. 2, pp. 485–497, Jun. 2010.

[21] L. Huang, M. Zhou, K. Hao, and E. Hou, "A survey of multi-robot regular and adversarial patrolling," *IEEE/CAA J. Automatica Sinica*, vol. 6, no. 4, pp. 894–903, Jul. 2019.

[22] Y. Wang et al., "Task offloading for post-disaster rescue in unmanned aerial vehicles networks," *IEEE/ACM Trans. Netw.*, vol. 30, no. 4, pp. 1525–1539, Aug. 2022.

[23] S. Al-Hussaini, J. M. Gregory, and S. K. Gupta, "Generating task reallocation suggestions to handle contingencies in human-supervised multi-robot missions," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 1, pp. 367–381, Jan. 2024.

[24] H. Osooli, P. Robinette, K. Jerath, and S. R. Ahmadzadeh, "A multi-robot task assignment framework for search and rescue with heterogeneous teams," in *Proc. IROS Adv. Multi-Agent Learn. - Coordination, Perception Control Workshop*, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2309.12589

[25] F. Zitouni, R. Maamri, and S. Harous, "FA–QABC–MRTA: A solution for solving the multi-robot task allocation problem," *Intell. Service Robot.*, vol. 12, no. 4, pp. 407–418, 2019.

[26] F. Zitouni, S. Harous, and R. Maamri, "Towards a distributed solution to multi-robot task allocation problem with energetic and spatiotemporal constraints," *Comput. Sci.*, vol. 21, pp. 3–24, 2020.

[27] W. Shen, L. Wang, and Q. Hao, "Agent-based distributed manufacturing process planning and scheduling: A state-of-the-art survey," *IEEE Trans. Syst., Man, Cybern., Part C (Appl. Rev.)*, vol. 36, no. 4, pp. 563–577, Jul. 2006.

[28] Y. Cao, W. Yu, W. Ren, and G. Chen, "An overview of recent progress in the study of distributed multi-agent coordination," *IEEE Trans. Ind. Informat.*, vol. 9, no. 1, pp. 427–438, Feb. 2013.

[29] S. Mayya, D. S. D'antonio, D. Saldaña, and V. Kumar, "Resilient task allocation in heterogeneous multi-robot systems," *IEEE Robot. Automat. Lett.*, vol. 6, no. 2, pp. 1327–1334, Apr. 2021.

[30] B. Fu, W. Smith, D. M. Rizzo, M. Castanier, M. Ghaffari, and K. Barton, "Robust task scheduling for heterogeneous robot teams under capability uncertainty," *IEEE Trans. Robot.*, vol. 39, no. 2, pp. 1087–1105, Apr. 2023.

[31] X. Luo and M. M. Zavlanos, "Temporal logic task allocation in heterogeneous multirobot systems," *IEEE Trans. Robot.*, vol. 38, no. 6, pp. 3602–3621, Dec. 2022.

[32] N. Zhao, H. Wu, and Y. Chen, "Coalition game-based computation resource allocation for wireless blockchain networks," *IEEE Internet Things J.*, vol. 6, no. 5, pp. 8507–8518, Oct. 2019.

[33] I. Jang, H. -S. Shin, and A. Tsourdos, "Anonymous hedonic game for task allocation in a large-scale multiple agent system," *IEEE Trans. Robot.*, vol. 34, no. 6, pp. 1534–1548, Dec. 2018.

[34] W. Saad, Z. Han, T. Basar, M. Debbah, and A. Hjorungnes, "Hedonic coalition formation for distributed task allocation among wireless agents," *IEEE Trans. Mobile Comput.*, vol. 10, no. 9, pp. 1327–1344, Sep. 2011.

[35] Y. Li, Z. Zhang, Z. He, and Q. Sun, "A heuristic task allocation method based on overlapping coalition formation game for heterogeneous UAVs," *IEEE Internet Things J.*, vol. 11, no. 17, pp. 28945–28959, Sep. 2024.

[36] F. Zitouni, S. Harous, and R. Maamri, "A distributed approach to the multi-robot task allocation problem using the consensus-based bundle algorithm and ant colony system," *IEEE Access*, vol. 8, pp. 27479–27494, 2020.

[37] Z. Chang, H. Deng, L. You, G. Min, S. Garg, and G. Kaddoum, "Trajectory design and resource allocation for multi-UAV networks: Deep reinforcement learning approaches," *IEEE Trans. Netw. Sci. Eng.*, vol. 10, no. 5, pp. 2940–2951, Sep./Oct. 2023.

[38] B. P. Gerkey and M. J. Matarić, "A formal analysis and taxonomy of task allocation in multi-robot systems," *Int. J. Robot. Res.*, vol. 23, no. 9, pp. 939–954, 2004.

[39] F. Zitouni, S. Harous, and R. Maamri, "A distributed solution to the multi-robot task allocation problem using ant colony optimization and bat algorithm," in *Proc. Adv. Mach. Learn. Comput. Intell.*, 2020, pp. 477–490.

[40] G. Peng, Y. Xia, X. Zhang, and L. Bai, "UAV-aided networks for emergency communications in areas with unevenly distributed users," *J. Commun. Inf. Netw.*, vol. 3, no. 4, pp. 23–32, 2018.

[41] D. Wu, X. Sun, and N. Ansari, "An FSO-based drone assisted mobile access network for emergency communications," *IEEE Trans. Netw. Sci. Eng.*, vol. 7, no. 3, pp. 1597–1606, Jul.–Sep. 2020.

[42] S. Huang, H. Yao, X. Wang, T. Mai, Z. Wang, and S. Guo, "Graph attentional based agglomerative cluster for UAV swarm networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 4, pp. 3311–3327, Jul./Aug. 2025.

[43] L. Luo, N. Chakraborty, and K. Sycara, "Distributed algorithms for multi-robot task assignment with task deadline constraints," *IEEE Trans. Autom. Sci. Eng.*, vol. 12, no. 3, pp. 876–888, Jul. 2015.

[44] Z. Yao, W. Cheng, W. Zhang, and H. Zhang, "Resource allocation for 5 G-UAV-based emergency wireless communications," *IEEE J. Sel. Areas Commun.*, vol. 39, no. 11, pp. 3395–3410, Nov. 2021.

[45] S. Qi, B. Lin, Y. Deng, X. Chen, and Y. Fang, "Minimizing maximum latency of task offloading for multi-UAV-assisted maritime search and rescue," *IEEE Trans. Veh. Technol.*, vol. 73, no. 9, pp. 13625–13638, Sep. 2024.

[46] W. Fan, X. Chun, Z. Fan, R. Zhang, S. Liu, and Y. Liu, "Dual-agent DRL-based service placement, task scheduling, and resource allocation for multi-sensor and multi-user edge computing networks," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 5, pp. 3416–3433, Sep./Oct. 2025.

[47] P. Qin, Y. Wang, J. Zhang, P. Li, and Y. Fu, "Multi-type disaster scenario task offloading in air-ground integrated search and rescue networks: A blockchain-assisted MFL approach," *IEEE Trans. Veh. Technol.*, early access, Jul. 2025, doi: 10.1109/TVT.2025.3587311.

[48] K. Ma, H. Zhao, J. Wang, Y. Wang, and Y. Shen, "A multi-UAV network formation scheme via integrated localization and motion planning," *IEEE Trans. Netw. Sci. Eng.*, vol. 12, no. 3, pp. 1552–1566, May/Jun. 2025.

[49] C. Toumieh and D. Floreano, "High-speed motion planning for aerial swarms in unknown and cluttered environments," *IEEE Trans. Robot.*, vol. 40, pp. 3642–3656, 2024.

[50] Y. Qin, Z. Wang, and H. Zhang, "Joint task allocation and power optimization in multi-UAV systems: An overlapping coalition formation game approach," *IEEE Trans. Veh. Technol.*, early access, Aug. 2025, doi: 10.1109/TVT.2025.3596944.

[51] Y. Yan, W. Bi, G. Ma, and A. Zhang, "Collaborative task allocation for large-scale heterogeneous AAV swarm: A hierarchical coalition formation game method," *IEEE Internet Things J.*, vol. 12, no. 14, pp. 27237–27254, Jul. 2025.

[52] N. Qi, Z. Huang, F. Zhou, Q. Shi, Q. Wu, and M. Xiao, "A task-driven sequential overlapping coalition formation game for resource allocation in heterogeneous uav networks," *IEEE Trans. Mobile Comput.*, vol. 22, no. 8, pp. 4439–4455, Aug. 2023.

[53] K. Wang, Y. Sun, P. Liu, Y. Zhang, and Z. Shao, "Energy-efficient deep reinforcement learning RSMA in multi-UAV-assisted wireless-powered communication network," *IEEE Trans. Netw. Sci. Eng.*, vol. 13, pp. 2420–2438, 2026.

[54] J. Zhang, Z. Wang, H. Wu, and W. Li, "Ordered submodularity-based value maximization of UAV data collection in earthquake areas," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 4886–4897, Sep./Oct. 2024.

[55] Y. Zhou, Z. Jin, H. Shi, L. Shi, N. Lu, and M. Dong, "Enhanced emergency communication services for post–disaster rescue: Multi-IRS assisted air-ground integrated data collection," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 5, pp. 4651–4664, Sep./Oct. 2024.

[56] P. Wang, C. Yu, M. Lv, and J. Cao, "Adaptive fixed-time optimal formation control for uncertain nonlinear multiagent systems using reinforcement learning," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 2, pp. 1729–1743, Mar./Apr. 2024.

[57] M. B. Dias, R. Zlot, N. Kalra, and A. Stentz, "Market-based multi-robot coordination: A survey and analysis," *Proc. IEEE*, vol. 94, no. 7, pp. 1257–1270, Jul. 2006.

[58] H. Wu, A. Ghadami, A. E. Bayrak, J. M. Smereka, and B. I. Epureanu, "Impact of heterogeneity and risk aversion on task allocation in multi-agent teams," *IEEE Robot. Automat. Lett.*, vol. 6, no. 4, pp. 7065–7072, Oct. 2021.

[59] X. Huo, H. Zhang, Z. Wang, C. Huang, and H. Yan, "A self-learning approach to heterogeneous multi-robot coalition formation under uncertainty," *IEEE Trans. Autom. Sci. Eng.*, vol. 22, pp. 3445–3457, 2025.

[60] J. Hu, P. Bhowmick, I. Jang, F. Arvin, and A. Lanzon, "A decentralized cluster formation containment framework for multirobot systems," *IEEE Trans. Robot.*, vol. 37, no. 6, pp. 1936–1955, Dec. 2021.

[61] J. Chen, X. Zhang, B. Xin, and H. Fang, "Coordination between unmanned aerial and ground vehicles: A taxonomy and optimization perspective," *IEEE Trans. Cybern.*, vol. 46, no. 4, pp. 959–972, Apr. 2016.

[62] C. Wei, Z. Ji, and B. Cai, "Particle swarm optimization for cooperative multi-robot task allocation: A multi-objective approach," *IEEE Robot. Automat. Lett.*, vol. 5, no. 2, pp. 2530–2537, Apr. 2020.

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.

[63] W. Guo, J. Li, G. Chen, Y. Niu, and C. Chen, "A PSO-optimized real-time fault-tolerant task allocation algorithm in wireless sensor networks," *IEEE Trans. Parallel Distrib. Syst.*, vol. 26, no. 12, pp. 3236–3249, Dec. 2015.
[64] J. Yang, H. Zhang, Y. Ling, C. Pan, and W. Sun, "Task allocation for wireless sensor network using modified binary particle swarm optimization," *IEEE Sensors J.*, vol. 14, no. 3, pp. 882–892, Mar. 2014.
[65] D. Zhu, H. Huang, and S. X. Yang, "Dynamic task assignment and path planning of multi-AUV system based on an improved self-organizing map and velocity synthesis method in three-dimensional underwater workspace," *IEEE Trans. Cybern.*, vol. 43, no. 2, pp. 504–514, Apr. 2013.
[66] A. Ajagekar, K. A. Hamoud, and F. You, "Hybrid classical-quantum optimization techniques for solving mixed-integer programming problems in production scheduling," *IEEE Trans. Quantum Eng.*, vol. 3, 2022, Art. no. 3102216.
[67] R. Zhang, Y. Feng, Y. Yang, and X. Li, "A deadlock-free hybrid estimation of distribution algorithm for cooperative multi-UAV task assignment with temporally coupled constraints," *IEEE Trans. Aerosp. Electron. Syst.*, vol. 59, no. 3, pp. 3329–3344, Jun. 2023.
[68] X. Huo, H. Zhang, C. Huang, Z. Wang, and H. Yan, "Task allocation with minimum requirements for multiple mobile robot systems: A game-theoretical approach," *IEEE Trans. Netw. Sci. Eng.*, vol. 11, no. 1, pp. 1202–1213, Jan./Feb. 2024.
[69] Z. Chen, L. Li, and Z. Kan, "Distributed task allocation and planning under temporal logic and communication constraints," *IEEE Robot. Automat. Lett.*, vol. 9, no. 7, pp. 6536–6543, Jul. 2024.
[70] S. Wang, Y. Liu, Y. Qiu, and J. Zhou, "Consensus-based decentralized task allocation for multi-agent systems and simultaneous multi-agent tasks," *IEEE Robot. Automat. Lett.*, vol. 7, no. 4, pp. 12593–12600, Oct. 2022.
[71] B. Zhou, H. Xu, and S. Shen, "RACER: Rapid collaborative exploration with a decentralized multi-UAV system," *IEEE Trans. Robot.*, vol. 39, no. 3, pp. 1816–1835, Jun. 2023.
[72] Q. Fang, J. Zhang, C. Wang, G. Xie, and C. Zhang, "Decentralized game-based auction algorithm for scheduling robotic chargers to service EVs with uncertain demands," *IEEE Trans. Intell. Veh.*, vol. 10, no. 2, pp. 886–899, Feb. 2025.
[73] M. G. Lagoudakis, M. Berhault, S. Koenig, P. Keskinocak, and A. J. Kleywegt, "Simple auctions with performance guarantees for multi-robot task allocation," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2004, pp. 698–705.
[74] M. Otte, M. J. Kuhlman, and D. Sofge, "Auctions for multi-robot task allocation in communication limited environments," *Auton. Robots*, vol. 44, pp. 547–584, 2020.
[75] K. Zhu, J. Tang, H. Guo, C. Gong, and J. Li, "Using a combinatorial auction-based approach for simulation of cooperative rescue operations in disaster relief," *Int. J. Modeling, Simulation, Sci. Comput.*, vol. 9, no. 04, 2018, Art. no. 1850035.
[76] X. Bai, A. Fielbaum, M. Kronmüller, L. Knoedler, and J. Alonso-Mora, "Group-based distributed auction algorithms for multi-robot task assignment," *IEEE Trans. Autom. Sci. Eng.*, vol. 20, no. 2, pp. 1292–1303, Apr. 2023.
[77] H. -L. Choi, L. Brunet, and J. P. How, "Consensus-based decentralized auctions for robust task allocation," *IEEE Trans. Robot.*, vol. 25, no. 4, pp. 912–926, Aug. 2009.
[78] W. Zhao, Q. Meng, and P. W. Chung, "A heuristic distributed task allocation method for multivehicle multitask problems and its application to search and rescue scenario," *IEEE Trans. Cybern.*, vol. 46, no. 4, pp. 902–915, Apr. 2016.
[79] A. Whitbrook, Q. Meng, and P. W. Chung, "Reliable, distributed scheduling and rescheduling for time-critical, multiagent systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 15, no. 2, pp. 732–747, Apr. 2018.
[80] J. Turner, Q. Meng, G. Schaefer, A. Whitbrook, and A. Soltoggio, "Distributed task rescheduling with time constraints for the optimization of total task allocations in a multirobot system," *IEEE Trans. Cybern.*, vol. 48, no. 9, pp. 2583–2597, Sep. 2018.
[81] X. Bai, C. Li, B. Zhang, Z. Wu, and S. S. Ge, "Efficient performance impact algorithms for multirobot task assignment with deadlines," *IEEE Trans. Ind. Electron.*, vol. 71, no. 11, pp. 14373–14382, Nov. 2024.
[82] J. Li, R. Chen, C. Wang, Y. Chen, Y. Huang, and X. Wang, "A performance-impact based multi-task distributed scheduling algorithm with task removal inference and deadlock avoidance," *Auton. Agents Multi-Agent Syst.*, vol. 37, no. 2, 2023, Art. no. 30.
[83] A. Whitbrook, Q. Meng, and P. W. H. Chung, "A novel distributed scheduling algorithm for time-critical multi-agent systems," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2015, pp. 6451–6458.
[84] S. Wang, Y. Liu, Y. Qiu, S. Li, and J. Zhou, "An efficient distributed task allocation method for maximizing task allocations of multirobot systems," *IEEE Trans. Autom. Sci. Eng.*, vol. 21, no. 3, pp. 3588–3602, Jul. 2024.



Yulin Li received the B.Eng. degree in automation from Beijing Information Science and Technology University, Beijing, China, in 2018, and the M.Eng. degree in control science and engineering from Bohai University, Jinzhou, China, in 2022. He is currently working toward the Ph.D. degree in control science and engineering with Nankai University, Tianjin, China. His research interests include adaptive control, game theory, multi-robot task allocation, and their applications in intelligent systems.



Chunyan Zhang (Member, IEEE) received the Ph.D. degree in general and fundamental mechanics from Peking University, Beijing, China, in 2013. From 2013 to 2014, she was a Postdoctoral Research Associate with the Faculty of Mathematics and Nature Sciences, Groningen University, Groningen, The Netherlands. She is currently a Professor with the College of Artificial Intelligence, Nankai University, Tianjin, China. Her research interests include evolutionary game dynamics, swarm intelligence in multiagent systems, and pattern recognition.



Qiuyang Fang received the B.S. degree in automatic control from Zhengzhou University, Zhengzhou, China, in 2019, the M.S. degree in control science and engineering from Northeastern University, Shenyang, China, in 2022. He is currently working toward the Ph.D. degree in artificial intelligence from Nankai University, Tianjin, China. His research interests include game theory, swarm intelligence, multi-robot task allocation, and distributed optimization.



Xudong Zhao (Senior Member, IEEE) received the B.S. degree in automation from the Harbin Institute of Technology, Harbin, China, in 2005, and the Ph.D. degree in control science and engineering from Space Control and Inertial Technology Center, Harbin Institute of Technology, in 2010. He was a Lecturer and an Associate Professor with the China University of Petroleum, Beijing, China. In 2013, he was with Bohai University, Jinzhou, China, as a Professor. In 2014, he was a Postdoctoral Fellow with the Department of Mechanical Engineering, The University of Hong Kong, Hong Kong. Since 2015, he has been with the Dalian University of Technology, Dalian, China, where he is currently a Professor. His research interests include hybrid systems, positive systems, multiagent systems, and control of aero engine. Dr. Zhao was awarded as the 2017–2022 Web of Science Highly Cited Researcher in Engineering. He is an Associate Editor for IEEE TRANSACTIONS ON SYSTEMS, MAN, AND CYBERNETICS: SYSTEMS, *Nonlinear Analysis: Hybrid Systems*, *Neurocomputing*, *International Journal of General Systems*, *ACTA Automatica Sinica*, *Assembly Automation*, and *Journal of Aeronautics*.



Jianlei Zhang (Member, IEEE) received the Ph.D. degree in general and fundamental mechanics from Peking University, Beijing, China, in 2014, and the Ph.D. degree in systems and control from the University of Groningen, Groningen, The Netherlands, in 2015. He is currently a Professor with the College of Artificial Intelligence, Nankai University, Tianjin, China. His research interests include evolutionary game theory, swarm intelligence, complex networks, pattern recognition, and evolutionary dynamics of collective behaviours.

Authorized licensed use limited to: Rajshahi University Of Engineering and Technology. Downloaded on June 04,2026 at 07:52:06 UTC from IEEE Xplore. Restrictions apply.